```python
#!/usr/bin/env python3
"""码极客具身导航多模态流式评测与诊断平台 V1.0 启动入口。

用法:
    python run.py                      # 默认 127.0.0.1:8080
    python run.py --port 9000
    python run.py --no-browser --reload
"""

from __future__ import annotations

import argparse
import os
import sys
import threading
import time
import webbrowser
from pathlib import Path

ROOT_DIR = Path(__file__).resolve().parent
if str(ROOT_DIR) not in sys.path:
    sys.path.insert(0, str(ROOT_DIR))


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="码极客具身导航多模态流式评测与诊断平台 V1.0")
    parser.add_argument("--host", default=None, help="监听地址，默认取配置文件")
    parser.add_argument("--port", type=int, default=None, help="监听端口，默认取配置文件")
    parser.add_argument("--config", default=None, help="配置文件路径")
    parser.add_argument("--no-browser", action="store_true", help="启动后不自动打开浏览器")
    parser.add_argument("--reload", action="store_true", help="开发模式，代码变更自动重载")
    return parser.parse_args()


def open_browser_later(url: str, delay: float = 1.5) -> None:
    def _open() -> None:
        time.sleep(delay)
        try:
            webbrowser.open(url)
        except Exception:  # noqa: BLE001 - 无图形环境时忽略
            pass

    threading.Thread(target=_open, daemon=True).start()


def main() -> None:
    args = parse_args()
    if args.config:
        os.environ["NAVEVAL_CONFIG"] = str(Path(args.config).resolve())
```

```
    # 读取配置需要先导入；bootstrap 在模块导入时完成
    from backend.app.core.config import load_settings, set_settings

    settings = load_settings(args.config)
    set_settings(settings)

    host = args.host or settings.server.host
    port = args.port or settings.server.port
    url = f"http://{'127.0.0.1' if host in ('0.0.0.0', '::') else host}:{port}"

    if not args.no_browser and settings.server.open_browser and not args.reload:
        open_browser_later(url)

    print("=" * 64)
    print(" 码极客具身导航多模态流式评测与诊断平台 V1.0")
    print(f" 访问地址：{url}")
    print(f" 接口文档：{url}/docs")
    print(f" 工作目录：{settings.storage.work_dir}")
    print("=" * 64)

    import uvicorn

    uvicorn.run(
        "backend.app.main:app",
        host=host,
        port=port,
        reload=args.reload,
        log_level=settings.logging.level.lower(),
        # 默认 5 s 太短：页面静一会儿后浏览器复用刚被服务端关掉的连接，
        # POST /api/sim/actions 会报 ERR_INVALID_HTTP_RESPONSE（浏览器不会重试 POST）
        timeout_keep_alive=30,
    )


if __name__ == "__main__":
    main()
"""应用入口：装配路由、异常处理与静态资源。"""

from __future__ import annotations

import asyncio
import logging
from contextlib import asynccontextmanager
from pathlib import Path

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
from fastapi.staticfiles import StaticFiles
```

```
from starlette.exceptions import HTTPException as StarletteHTTPException

from .api import (
    analysis,
    audit,
    auth,
    datasets,
    imports,
    models,
    overview,
    replay,
    reports,
    sim,
    splits,
    system,
    tasks,
    templates,
    users,
)
from .core.config import get_settings
from .core.events import BUS
from .core.logging import setup_logging
from .core.response import (
    INTERNAL_ERROR,
    NOT_FOUND,
    PARAM_INVALID,
    UNAUTHORIZED,
    AppError,
)
from .core.security import init_secret, set_iterations
from .db.seed import seed_all
from .db.session import init_database, session_scope
from .services.backup_auto_importer import get_auto_importer
from .services.scheduler import get_scheduler
from .services.sim import sim_session

logger = logging.getLogger(__name__)

FRONTEND_DIST = Path(__file__).resolve().parents[2] / "frontend" / "dist"

PLACEHOLDER_HTML = """<!doctype html>
<html lang="zh-CN"><head><meta charset="utf-8"><title>码极客具身导航多模态流式评测与诊断平台 V1.0</title>
<style>body{font-family:system-ui,-apple-system,"Segoe UI",sans-serif;background:#0f172a;color:#e2e8f0;
display:flex;align-items:center;justify-content:center;height:100vh;margin:0}
.box{max-width:640px;padding:40px;border:1px solid #1e293b;border-radius:14px;background:#111c33}
h1{font-size:20px;margin:0 0 12px}p{color:#94a3b8;line-height:1.8;margin:6px 0}
code{background:#1e293b;padding:2px 6px;border-radius:4px;color:#7dd3fc}</style></head>
<body><div class="box"><h1>码极客具身导航多模态流式评测与诊断平台 V1.0</h1>
<p>服务端已启动，前端构建产物尚未生成。</p>
<p>开发模式：<code>cd frontend &amp;&amp; npm install &amp;&amp; npm run dev</code></p>
```

```
<p>生产模式：<code>cd frontend &amp;&amp; npm run build</code> 后刷新本页。</p>
<p>接口文档：<a style="color:#7dd3fc" href="/docs">/docs</a></p></div></body></html>"""


def bootstrap() -> None:
    """初始化配置、日志、密钥与数据库，并写入种子数据。"""
    settings = get_settings()
    setup_logging(settings)
    init_secret(settings.secret_file)
    set_iterations(settings.security.pbkdf2_iterations)
    first_time = init_database(settings)
    with session_scope() as session:
        summary = seed_all(session, settings)
    logger.info(
        "初始化完成 first_time=%s 种子=%s 工作目录=%s",
        first_time,
        summary,
        settings.storage.work_dir,
    )


@asynccontextmanager
async def lifespan(app: FastAPI):
    BUS.bind_loop(asyncio.get_running_loop())
    scheduler = get_scheduler()
    await scheduler.start()
    app.state.scheduler = scheduler
    auto_importer = get_auto_importer()
    await auto_importer.start()
    app.state.backup_auto_importer = auto_importer
    logger.info("服务已启动")
    yield
    await auto_importer.shutdown()
    await scheduler.shutdown()
    # 仿真线程持有可能残留的 headless GL context，必须在进程退出前串行释放
    await asyncio.to_thread(sim_session.shutdown)
    logger.info("服务已停止")


def create_app() -> FastAPI:
    bootstrap()

    app = FastAPI(
        title="码极客具身导航多模态流式评测与诊断平台",
        version="1.0.0",
        description="码极客 NavEval Studio 服务端接口",
        lifespan=lifespan,
    )

    app.add_middleware(
```

```
        CORSMiddleware,
        allow_origins=["http://localhost:5173", "http://127.0.0.1:5173"],
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )

    for module in (
        auth,
        users,
        system,
        overview,
        audit,
        datasets,
        splits,
        imports,
        models,
        templates,
        tasks,
        analysis,
        reports,
        replay,
        sim,
    ):
        app.include_router(module.router, prefix="/api")

    @app.get("/health")
    def health() -> dict:
        return {"status": "ok", "version": "1.0.0"}

    @app.exception_handler(AppError)
    async def handle_app_error(_: Request, exc: AppError) -> JSONResponse:
        return JSONResponse(
            status_code=exc.http_status,
            content={"code": exc.code, "message": exc.message, "data": None},
        )

    @app.exception_handler(RequestValidationError)
    async def handle_validation_error(_: Request, exc: RequestValidationError) -> JSONResponse:
        first = exc.errors()[0] if exc.errors() else {}
        field = ".".join(str(part) for part in first.get("loc", [])[1:]) or "参数"
        return JSONResponse(
            status_code=400,
            content={
                "code": PARAM_INVALID,
                "message": f"{field}: {first.get('msg', '参数不合法')}",
                "data": {"errors": exc.errors()},
            },
        )
```

```
251     @app.exception_handler(StarletteHTTPException)
252     async def handle_http_error(_: Request, exc: StarletteHTTPException) -> JSONResponse:
253         code = UNAUTHORIZED if exc.status_code == 401 else NOT_FOUND
254         return JSONResponse(
255             status_code=exc.status_code,
256             content={"code": code if exc.status_code < 500 else INTERNAL_ERROR,
257                      "message": str(exc.detail), "data": None},
258         )

260     @app.exception_handler(Exception)
261     async def handle_unexpected(request: Request, exc: Exception) -> JSONResponse:
262         logger.exception("未处理异常 %s %s", request.method, request.url.path)
263         return JSONResponse(
264             status_code=500,
265             content={"code": INTERNAL_ERROR, "message": "服务内部错误", "data": None},
266         )

268     _mount_frontend(app)
269     return app


272 def _mount_frontend(app: FastAPI) -> None:
273     index_file = FRONTEND_DIST / "index.html"
274     assets_dir = FRONTEND_DIST / "assets"

276     if assets_dir.exists():
277         app.mount("/assets", StaticFiles(directory=assets_dir), name="assets")

279     if index_file.exists():

281         @app.get("/{full_path:path}", include_in_schema=False)
282         async def spa(full_path: str):
283             candidate = (FRONTEND_DIST / full_path).resolve()
284             if (
285                 full_path
286                 and FRONTEND_DIST.resolve() in candidate.parents
287                 and candidate.is_file()
288             ):
289                 return FileResponse(candidate)
290             # 入口 HTML 必须不缓存：否则前端重新构建后，浏览器仍用旧 HTML 去加载已删除的 chunk
291             return FileResponse(
292                 index_file,
293                 headers={"Cache-Control": "no-cache, no-store, must-revalidate"},
294             )

296     else:

298         @app.get("/", include_in_schema=False)
299         async def placeholder():
300             return HTMLResponse(PLACEHOLDER_HTML)
```

```
app = create_app()
"""配置加载。

优先级：环境变量 > config/default.yaml > 内置默认值。
所有相对路径以仓库根目录为基准解析，避免工作目录不同导致路径漂移。
"""

from __future__ import annotations

import copy
import os
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any

import yaml

ROOT_DIR = Path(__file__).resolve().parents[3]
CONFIG_DIR = ROOT_DIR / "config"

DEFAULT_CONFIG: dict[str, Any] = {
    "server": {"host": "0.0.0.0", "port": 8080, "open_browser": False},
    "storage": {
        "work_dir": "./storage",
        "db_path": "./storage/eval.db",
        "artifact_keep_days": 90,
    },
    "scheduler": {
        "max_parallel_tasks": 2,
        "episode_timeout_s": 300,
        "poll_interval_ms": 500,
        "gpu_ids": [],
    },
    "security": {"token_ttl_hours": 12, "pbkdf2_iterations": 200000},
    "logging": {"level": "INFO", "keep_days": 30},
    "demo": {"enabled": True, "sample_results_dir": "./storage/sample_results"},
    "import": {
        "results_root": "",
        "backup_root": "",
        "auto_enabled": True,
        "auto_interval_s": 300,
        "auto_sections": ["datasets", "scenes", "models", "results", "assets"],
        "auto_scan_episodes": True,
        "scan_limit_mb": 32,
        "scan_exclude": ["train", "envdrop", "scalevln"],
        "running_grace_s": 600,
    },
    "sim": {
```

```
        "scene_root": "",
        "gpu_ids": [0],
        "tick_hz": 4.0,
        "render_hz": 15.0,
        "stream_hz": 20.0,
        "jpeg_quality": 82,
        "max_clients": 4,
        "idle_timeout_s": 900,
        "free_mem_warn_mb": 4096,
        "pip": True,
        "record_fps": 10.0,
        "record_max_frames": 6000,
        "robot_mesh": "go2",
        "robot_scale": 1.0,
        "robot_yaw_deg": 0.0,
    },
    "policy": {
        "enabled": True,
        "model_profile": "",
        "gpu_id": None,
        "min_free_mem_mb": 12000,
        "num_frames": 8,
        "num_history": 8,
        "num_future_steps": 4,
        "infer_timeout_s": 30,
        "idle_pause_s": 60,
    },
}

ALL_IMPORT_SECTIONS = ("datasets", "scenes", "models", "results", "assets")


@dataclass
class ServerConfig:
    host: str = "0.0.0.0"
    port: int = 8080
    open_browser: bool = False


@dataclass
class StorageConfig:
    work_dir: Path = ROOT_DIR / "storage"
    db_path: Path = ROOT_DIR / "storage" / "eval.db"
    artifact_keep_days: int = 90


@dataclass
class SchedulerConfig:
    max_parallel_tasks: int = 2
    episode_timeout_s: int = 300
```

```
    poll_interval_ms: int = 500
    gpu_ids: list[str] = field(default_factory=list)


@dataclass
class SecurityConfig:
    token_ttl_hours: int = 12
    pbkdf2_iterations: int = 200000


@dataclass
class LoggingConfig:
    level: str = "INFO"
    keep_days: int = 30


@dataclass
class DemoConfig:
    enabled: bool = True
    sample_results_dir: Path = ROOT_DIR / "storage" / "sample_results"


@dataclass
class ImportConfig:
    """历史评测产物与备份库自动导入的默认绑定。"""

    results_root: Path = ROOT_DIR / "storage" / "sample_results"
    backup_root: Path = ROOT_DIR / "storage" / "sample_results"
    auto_enabled: bool = True
    auto_interval_s: int = 300
    auto_sections: list[str] = field(default_factory=lambda: list(ALL_IMPORT_SECTIONS))
    auto_scan_episodes: bool = True
    scan_limit_mb: int = 32
    scan_exclude: list[str] = field(default_factory=lambda: ["train", "envdrop", "scalevln"])
    running_grace_s: int = 600


@dataclass
class SimConfig:
    """交互式仿真: 场景根目录与渲染/会话参数 (全部有默认值, 缺省也能启动) 。"""

    scene_root: Path | None = None
    gpu_ids: list[int] = field(default_factory=lambda: [0])
    tick_hz: float = 4.0
    render_hz: float = 15.0
    stream_hz: float = 20.0
    jpeg_quality: int = 82
    max_clients: int = 4
    idle_timeout_s: int = 900
    free_mem_warn_mb: int = 4096
```

```
    # 第三视角：多挂一路 320×240 相机，画面里做成第一视角右上角的画中画。
    # 实测默认 15 fps 上限下开与不开都是 14.4 fps（裸渲染 50 vs 55 fps，差异在噪声内）
    pip: bool = True
    # 会话录制：落帧采样率（0 表示不录制），超过最大帧数后丢帧
    record_fps: float = 10.0
    record_max_frames: int = 6000
    # 机器狗网格："" / "none" = 不显示；"go2" = 内置模型；也可给 .object_config.json
    # 或 .glb 的绝对路径。只在画中画里能看到，第一视角视锥看不到它（见 docs/09 #9）
    robot_mesh: str = "go2"
    robot_scale: float = 1.0
    robot_yaw_deg: float = 0.0


@dataclass
class PolicyConfig:
    """模型驾驶：一次 forward 出几个动作、显存门槛、超时。

    全部有默认值——`enabled: false` 时接口直接拒绝，页面也不会显示入口，
    手动驾驶完全不受影响。
    """

    enabled: bool = True
    model_profile: str = ""
    gpu_id: int | None = None
    min_free_mem_mb: int = 12000
    num_frames: int = 8
    num_history: int = 8
    num_future_steps: int = 4
    infer_timeout_s: float = 30.0
    idle_pause_s: float = 60.0


@dataclass
class Settings:
    """进程内全局配置。"""

    root: Path
    server: ServerConfig
    storage: StorageConfig
    scheduler: SchedulerConfig
    security: SecurityConfig
    logging: LoggingConfig
    demo: DemoConfig
    imports: ImportConfig = field(default_factory=ImportConfig)
    sim: SimConfig = field(default_factory=SimConfig)
    policy: PolicyConfig = field(default_factory=PolicyConfig)
    metrics: dict[str, Any] = field(default_factory=dict)
    models: dict[str, Any] = field(default_factory=dict)
    raw: dict[str, Any] = field(default_factory=dict)
```

```
    # ---- 常用派生路径 ----
    @property
    def runs_dir(self) -> Path:
        return self.storage.work_dir / "runs"

    @property
    def reports_dir(self) -> Path:
        return self.storage.work_dir / "reports"

    @property
    def cache_dir(self) -> Path:
        return self.storage.work_dir / "cache"

    @property
    def logs_dir(self) -> Path:
        return self.storage.work_dir / "logs"

    @property
    def secret_file(self) -> Path:
        return self.storage.work_dir / "secret.key"

    def ensure_dirs(self) -> None:
        for path in (
            self.storage.work_dir,
            self.runs_dir,
            self.reports_dir,
            self.cache_dir,
            self.logs_dir,
            self.storage.db_path.parent,
        ):
            path.mkdir(parents=True, exist_ok=True)


def _deep_merge(base: dict, override: dict) -> dict:
    result = copy.deepcopy(base)
    for key, value in override.items():
        if isinstance(value, dict) and isinstance(result.get(key), dict):
            result[key] = _deep_merge(result[key], value)
        else:
            result[key] = value
    return result


def _resolve(path_value: str | Path) -> Path:
    path = Path(path_value)
    if not path.is_absolute():
        path = (ROOT_DIR / path).resolve()
    return path
```

```
551 def _load_yaml(path: Path) -> dict[str, Any]:
552     if not path.exists():
553         return {}
554     with path.open("r", encoding="utf-8") as handle:
555         return yaml.safe_load(handle) or {}


558 def _load_import_config(data: dict[str, Any]) -> ImportConfig:
559     """组装导入配置；未单独配置备份根目录时沿用历史产物根目录。"""
560     raw = data.get("import", {}) or {}
561     fallback = raw.get("results_root") or data["demo"]["sample_results_dir"]
562     sections = [str(item) for item in raw.get("auto_sections", ALL_IMPORT_SECTIONS)]
563     sections = [item for item in sections if item in ALL_IMPORT_SECTIONS]
564     return ImportConfig(
565         results_root=_resolve(fallback),
566         backup_root=_resolve(raw.get("backup_root") or fallback),
567         auto_enabled=bool(raw.get("auto_enabled", True)),
568         auto_interval_s=max(10, int(raw.get("auto_interval_s", 300))),
569         auto_sections=sections or list(ALL_IMPORT_SECTIONS),
570         auto_scan_episodes=bool(raw.get("auto_scan_episodes", True)),
571         scan_limit_mb=max(1, int(raw.get("scan_limit_mb", 32))),
572         scan_exclude=[str(item).lower() for item in raw.get("scan_exclude", ["train", "envdrop", "scalevln"])],
573         running_grace_s=max(30, int(raw.get("running_grace_s", 600))),
574     )


577 def _load_sim_config(data: dict[str, Any]) -> SimConfig:
578     """组装仿真配置；场景根目录留空时由 episode 解析按数据集元数据回退。"""
579     raw = data.get("sim", {}) or {}
580     scene_root = str(raw.get("scene_root") or "").strip()
581     gpu_ids: list[int] = []
582     for item in raw.get("gpu_ids") or [0]:
583         try:
584             gpu_ids.append(int(item))
585         except (TypeError, ValueError):
586             continue
587     return SimConfig(
588         scene_root=_resolve(scene_root) if scene_root else None,
589         gpu_ids=gpu_ids or [0],
590         tick_hz=float(raw.get("tick_hz", 4.0)),
591         render_hz=float(raw.get("render_hz", 15.0)),
592         stream_hz=float(raw.get("stream_hz", 20.0)),
593         jpeg_quality=max(40, min(98, int(raw.get("jpeg_quality", 82)))),
594         max_clients=max(1, int(raw.get("max_clients", 4))),
595         idle_timeout_s=max(0, int(raw.get("idle_timeout_s", 900))),
596         free_mem_warn_mb=max(0, int(raw.get("free_mem_warn_mb", 4096))),
597         pip=bool(raw.get("pip", True)),
598         record_fps=max(0.0, min(30.0, float(raw.get("record_fps", 10.0)))),
599         record_max_frames=max(0, int(raw.get("record_max_frames", 6000))),
600         robot_mesh=str(raw.get("robot_mesh", "go2") or "").strip(),
```

```
        robot_scale=max(0.1, min(5.0, float(raw.get("robot_scale", 1.0)))),
        robot_yaw_deg=float(raw.get("robot_yaw_deg", 0.0)),
    )


def _load_policy_config(data: dict[str, Any]) -> PolicyConfig:
    """组装模型驾驶配置；gpu_id 留空表示自动挑空闲显存最多的一张卡。"""
    raw = data.get("policy", {}) or {}
    gpu_raw = raw.get("gpu_id", None)
    gpu_id: int | None = None
    if gpu_raw is not None and str(gpu_raw).strip() != "":
        try:
            gpu_id = int(gpu_raw)
        except (TypeError, ValueError):
            gpu_id = None
    return PolicyConfig(
        enabled=bool(raw.get("enabled", True)),
        model_profile=str(raw.get("model_profile") or "").strip(),
        gpu_id=gpu_id,
        min_free_mem_mb=max(0, int(raw.get("min_free_mem_mb", 12000))),
        num_frames=max(1, int(raw.get("num_frames", 8))),
        num_history=max(1, int(raw.get("num_history", 8))),
        num_future_steps=max(1, int(raw.get("num_future_steps", 4))),
        infer_timeout_s=float(raw.get("infer_timeout_s", 30)),
        idle_pause_s=float(raw.get("idle_pause_s", 60)),
    )


def load_settings(config_path: str | Path | None = None) -> Settings:
    """读取配置并组装为 Settings。"""
    path = Path(config_path) if config_path else Path(
        os.environ.get("NAVEVAL_CONFIG", CONFIG_DIR / "default.yaml")
    )
    data = _deep_merge(DEFAULT_CONFIG, _load_yaml(path))

    # 环境变量覆盖，便于容器化部署
    if os.environ.get("NAVEVAL_HOST"):
        data["server"]["host"] = os.environ["NAVEVAL_HOST"]
    if os.environ.get("NAVEVAL_PORT"):
        data["server"]["port"] = int(os.environ["NAVEVAL_PORT"])
    if os.environ.get("NAVEVAL_WORK_DIR"):
        data["storage"]["work_dir"] = os.environ["NAVEVAL_WORK_DIR"]
    if os.environ.get("NAVEVAL_IMPORT_ROOT"):
        data["import"]["backup_root"] = os.environ["NAVEVAL_IMPORT_ROOT"]

    storage = data["storage"]
    storage_cfg = StorageConfig(
        work_dir=_resolve(storage["work_dir"]),
        db_path=_resolve(storage["db_path"]),
        artifact_keep_days=int(storage.get("artifact_keep_days", 90)),
```

```
    )

    settings = Settings(
        root=ROOT_DIR,
        server=ServerConfig(**data["server"]),
        storage=storage_cfg,
        scheduler=SchedulerConfig(
            max_parallel_tasks=int(data["scheduler"]["max_parallel_tasks"]),
            episode_timeout_s=int(data["scheduler"]["episode_timeout_s"]),
            poll_interval_ms=int(data["scheduler"]["poll_interval_ms"]),
            gpu_ids=[str(item) for item in data["scheduler"].get("gpu_ids", [])],
        ),
        security=SecurityConfig(**data["security"]),
        logging=LoggingConfig(**data["logging"]),
        demo=DemoConfig(
            enabled=bool(data["demo"]["enabled"]),
            sample_results_dir=_resolve(data["demo"]["sample_results_dir"]),
        ),
        imports=_load_import_config(data),
        sim=_load_sim_config(data),
        policy=_load_policy_config(data),
        metrics=_load_yaml(CONFIG_DIR / "metrics.yaml"),
        models=_load_yaml(CONFIG_DIR / "models.yaml"),
        raw=data,
    )
    settings.ensure_dirs()
    return settings


_settings: Settings | None = None


def get_settings() -> Settings:
    global _settings
    if _settings is None:
        _settings = load_settings()
    return _settings


def set_settings(settings: Settings) -> None:
    """测试与启动脚本可通过该入口注入配置。"""
    global _settings
    _settings = settings
"""口令哈希与访问令牌。

口令: PBKDF2-HMAC-SHA256, 随机 16 字节盐, 迭代次数由配置决定。
令牌: base64url(payload).base64url(hmac_sha256(payload)), 无第三方依赖。
密钥首次启动生成并保存到 storage/secret.key, 重启后令牌仍有效。
"""
```

```python
from __future__ import annotations

import base64
import hashlib
import hmac
import json
import os
import secrets
import time
from pathlib import Path

from .response import UNAUTHORIZED, AppError

_SECRET: bytes | None = None
_ITERATIONS = 200000


def init_secret(secret_file: Path) -> bytes:
    """初始化签名密钥，文件不存在则生成。"""
    global _SECRET
    if secret_file.exists():
        _SECRET = secret_file.read_bytes()
    else:
        secret_file.parent.mkdir(parents=True, exist_ok=True)
        _SECRET = secrets.token_bytes(32)
        secret_file.write_bytes(_SECRET)
        try:
            os.chmod(secret_file, 0o600)
        except OSError:
            pass
    return _SECRET


def set_iterations(iterations: int) -> None:
    global _ITERATIONS
    _ITERATIONS = max(10000, int(iterations))


def _secret() -> bytes:
    if _SECRET is None:
        raise RuntimeError("签名密钥尚未初始化")
    return _SECRET


def hash_password(password: str, salt: bytes | None = None) -> tuple[str, str]:
    salt = salt or secrets.token_bytes(16)
    digest = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, _ITERATIONS)
    return digest.hex(), salt.hex()
```

```
751 def verify_password(password: str, password_hash: str, salt_hex: str) -> bool:
752     try:
753         salt = bytes.fromhex(salt_hex)
754     except ValueError:
755         return False
756     digest = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, _ITERATIONS)
757     return hmac.compare_digest(digest.hex(), password_hash)


760 def _b64encode(raw: bytes) -> str:
761     return base64.urlsafe_b64encode(raw).rstrip(b"=").decode("ascii")


764 def _b64decode(text: str) -> bytes:
765     padding = "=" * (-len(text) % 4)
766     return base64.urlsafe_b64decode(text + padding)


769 def create_token(user_id: int, username: str, role: str, ttl_hours: int) -> str:
770     now = int(time.time())
771     payload = {
772         "uid": user_id,
773         "username": username,
774         "role": role,
775         "iat": now,
776         "exp": now + ttl_hours * 3600,
777     }
778     body = _b64encode(json.dumps(payload, separators=(",", ":")).encode("utf-8"))
779     signature = hmac.new(_secret(), body.encode("ascii"), hashlib.sha256).digest()
780     return f"{body}.{_b64encode(signature)}"


783 def decode_token(token: str) -> dict:
784     """校验并解析令牌，失败抛 1002。"""
785     try:
786         body, signature = token.split(".", 1)
787     except ValueError as exc:
788         raise AppError(UNAUTHORIZED, "令牌格式错误") from exc

790     expected = hmac.new(_secret(), body.encode("ascii"), hashlib.sha256).digest()
791     try:
792         provided = _b64decode(signature)
793     except Exception as exc:  # noqa: BLE001 - 解码失败一律视为非法令牌
794         raise AppError(UNAUTHORIZED, "令牌签名非法") from exc
795     if not hmac.compare_digest(expected, provided):
796         raise AppError(UNAUTHORIZED, "令牌签名校验失败")

798     try:
799         payload = json.loads(_b64decode(body))
800     except Exception as exc:  # noqa: BLE001
```

```
        raise AppError(UNAUTHORIZED, "令牌内容损坏") from exc

    if int(payload.get("exp", 0)) < int(time.time()):
        raise AppError(UNAUTHORIZED, "登录已过期，请重新登录")
    return payload
"""统一响应结构与业务异常。

所有接口返回 {"code": 0, "message": "ok", "data": ...};
异常通过 AppError 抛出，由 main.py 的异常处理器转换为对应 HTTP 状态。
"""

from __future__ import annotations

from typing import Any

# 业务错误码
OK = 0
PARAM_INVALID = 1001
UNAUTHORIZED = 1002
FORBIDDEN = 1003
NOT_FOUND = 1004
CONFLICT = 1005
CAUSAL_VIOLATION = 1006
INTERNAL_ERROR = 2001
BACKEND_UNAVAILABLE = 2002

DEFAULT_STATUS = {
    PARAM_INVALID: 400,
    UNAUTHORIZED: 401,
    FORBIDDEN: 403,
    NOT_FOUND: 404,
    CONFLICT: 409,
    CAUSAL_VIOLATION: 422,
    INTERNAL_ERROR: 500,
    BACKEND_UNAVAILABLE: 503,
}


class AppError(Exception):
    """可预期的业务异常。"""

    def __init__(self, code: int, message: str, http_status: int | None = None):
        super().__init__(message)
        self.code = code
        self.message = message
        self.http_status = http_status or DEFAULT_STATUS.get(code, 400)


def ok(data: Any = None, message: str = "ok") -> dict[str, Any]:
    return {"code": OK, "message": message, "data": data}
```

```
def fail(code: int, message: str, http_status: int | None = None) -> dict[str, Any]:
    return {
        "code": code,
        "message": message,
        "data": None,
        "http_status": http_status or DEFAULT_STATUS.get(code, 400),
    }


def page(items: list[Any], total: int, page_no: int, page_size: int) -> dict[str, Any]:
    return {
        "items": items,
        "total": total,
        "page": page_no,
        "page_size": page_size,
    }
"""进程内事件总线。

任务执行线程 / 协程通过 publish 推送进度、资源采样与日志;
SSE 接口通过 subscribe 获取队列，逐个取出并以 text/event-stream 下发。
单机单进程模型下无需 Redis，保持零外部依赖。
"""

from __future__ import annotations

import asyncio
from collections import defaultdict
from typing import Any


class EventBus:
    def __init__(self, queue_size: int = 256) -> None:
        self._channels: dict[int, set[asyncio.Queue]] = defaultdict(set)
        self._queue_size = queue_size
        self._loop: asyncio.AbstractEventLoop | None = None

    def bind_loop(self, loop: asyncio.AbstractEventLoop) -> None:
        """绑定主事件循环，供工作线程安全投递。"""
        self._loop = loop

    def subscribe(self, task_id: int) -> asyncio.Queue:
        queue: asyncio.Queue = asyncio.Queue(maxsize=self._queue_size)
        self._channels[task_id].add(queue)
        return queue

    def unsubscribe(self, task_id: int, queue: asyncio.Queue) -> None:
        channel = self._channels.get(task_id)
        if channel:
```

```
            channel.discard(queue)
            if not channel:
                self._channels.pop(task_id, None)

    def publish(self, task_id: int, event: str, data: dict[str, Any]) -> None:
        message = {"event": event, "data": data}
        for queue in list(self._channels.get(task_id, ())):
            self._put(queue, message)

    def publish_threadsafe(self, task_id: int, event: str, data: dict[str, Any]) -> None:
        """从子线程投递，转交主循环执行。"""
        if self._loop is None or not self._loop.is_running():
            self.publish(task_id, event, data)
            return
        self._loop.call_soon_threadsafe(self.publish, task_id, event, data)

    def _put(self, queue: asyncio.Queue, message: dict[str, Any]) -> None:
        try:
            queue.put_nowait(message)
        except asyncio.QueueFull:
            # 消费端过慢时丢弃最旧事件，保证生产者不被阻塞
            try:
                queue.get_nowait()
            except asyncio.QueueEmpty:
                pass
            try:
                queue.put_nowait(message)
            except asyncio.QueueFull:
                pass


BUS = EventBus()
"""数据库引擎与会话。

建库使用 schema.sql 一次性执行，运行时由 SQLAlchemy 管理连接。
SQLite 开启 WAL 与外键约束，超时 30 秒以吸收并发写。
"""

from __future__ import annotations

import sqlite3
from contextlib import contextmanager
from pathlib import Path
from typing import Iterator

from sqlalchemy import Engine, create_engine, event, text
from sqlalchemy.orm import Session, sessionmaker

from ..core.config import Settings, get_settings
```

```
SCHEMA_FILE = Path(__file__).with_name("schema.sql")
SCHEMA_VERSION = 1

_engine: Engine | None = None
_session_factory: sessionmaker | None = None


def _schema_applied(db_path: Path) -> bool:
    if not db_path.exists():
        return False
    try:
        with sqlite3.connect(db_path) as conn:
            row = conn.execute(
                "SELECT name FROM sqlite_master WHERE type='table' AND name='schema_version'"
            ).fetchone()
            return row is not None
    except sqlite3.DatabaseError:
        return False


def _apply_schema(db_path: Path) -> None:
    db_path.parent.mkdir(parents=True, exist_ok=True)
    sql = SCHEMA_FILE.read_text(encoding="utf-8")
    with sqlite3.connect(db_path) as conn:
        conn.executescript(sql)
        conn.execute(
            "INSERT OR IGNORE INTO schema_version (version, description) VALUES (?, ?)",
            (SCHEMA_VERSION, "initial schema"),
        )
        conn.commit()


def configure(settings: Settings | None = None) -> Engine:
    """构建引擎（幂等）。"""
    global _engine, _session_factory
    settings = settings or get_settings()
    if _engine is not None:
        return _engine

    db_path = settings.storage.db_path
    db_path.parent.mkdir(parents=True, exist_ok=True)
    engine = create_engine(
        f"sqlite:///{db_path}",
        future=True,
        connect_args={"check_same_thread": False, "timeout": 30},
    )

    @event.listens_for(engine, "connect")
    def _set_pragmas(dbapi_connection, _record):  # noqa: ANN001
        cursor = dbapi_connection.cursor()
```

```
        cursor.execute("PRAGMA journal_mode=WAL")
        cursor.execute("PRAGMA foreign_keys=ON")
        cursor.execute("PRAGMA busy_timeout=30000")
        cursor.close()

    _engine = engine
    _session_factory = sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)
    return engine


def get_engine() -> Engine:
    if _engine is None:
        return configure()
    return _engine


def get_session_factory() -> sessionmaker:
    if _session_factory is None:
        configure()
    assert _session_factory is not None
    return _session_factory


@contextmanager
def session_scope() -> Iterator[Session]:
    """供服务层使用的会话上下文，异常自动回滚。"""
    session = get_session_factory()()
    try:
        yield session
        session.commit()
    except Exception:
        session.rollback()
        raise
    finally:
        session.close()


def get_db() -> Iterator[Session]:
    """FastAPI 依赖。"""
    session = get_session_factory()()
    try:
        yield session
        session.commit()
    except Exception:
        session.rollback()
        raise
    finally:
        session.close()
```

```
def init_database(settings: Settings | None = None) -> bool:
    """建库并执行迁移，返回是否为首次创建。"""
    settings = settings or get_settings()
    settings.ensure_dirs()
    db_path = settings.storage.db_path
    first_time = not _schema_applied(db_path)
    if first_time:
        _apply_schema(db_path)
    configure(settings)
    _migrate()
    return first_time


def _migrate() -> None:
    """按 schema_version 顺序执行补丁脚本（当前仅初始版本）。"""
    migrations_dir = Path(__file__).with_name("migrations")
    if not migrations_dir.exists():
        return

    with session_scope() as session:
        current = session.execute(text("SELECT COALESCE(MAX(version), 0) FROM schema_version")).scalar()

    for script in sorted(migrations_dir.glob("*.sql")):
        try:
            version = int(script.stem.split("_", 1)[0])
        except ValueError:
            continue
        if version <= current:
            continue
        with sqlite3.connect(get_settings().storage.db_path) as conn:
            conn.executescript(script.read_text(encoding="utf-8"))
            conn.execute(
                "INSERT OR IGNORE INTO schema_version (version, description) VALUES (?, ?)",
                (version, script.stem),
            )
            conn.commit()


def reset_engine() -> None:
    """测试用：释放引擎，下次访问重新构建。"""
    global _engine, _session_factory
    if _engine is not None:
        _engine.dispose()
    _engine = None
    _session_factory = None
"""ORM 映射汇总。

导入本模块即完成所有表的注册，Base.metadata 可直接用于建表或校验。
"""
```

```
from .backup import BackupAsset
from .dataset import Dataset, Episode, Split, SplitEpisode
from .system import SchemaVersion, Setting
from .task import Artifact, ConfigTemplate, EvalTask, ModelProfile, ResourceSample, TaskEpisode
from .user import AppUser, AuditLog

__all__ = [
    "BackupAsset",
    "AppUser",
    "AuditLog",
    "Dataset",
    "Episode",
    "Split",
    "SplitEpisode",
    "ConfigTemplate",
    "ModelProfile",
    "EvalTask",
    "TaskEpisode",
    "ResourceSample",
    "Artifact",
    "Setting",
    "SchemaVersion",
]
"""配置模板、模型档案、评测任务、结果明细与产物映射。"""

from __future__ import annotations

from sqlalchemy import Column, Float, ForeignKey, Integer, String, Text

from ..db.base import Base


class ConfigTemplate(Base):
    __tablename__ = "config_template"

    id = Column(Integer, primary_key=True, autoincrement=True)
    name = Column(String, nullable=False, unique=True)
    modality = Column(String, nullable=False, default="vision")
    memory_policy = Column(String, nullable=False, default="fixed_window")
    params_json = Column(Text, nullable=False, default="{}")
    description = Column(Text, nullable=False, default="")
    builtin = Column(Integer, nullable=False, default=0)
    created_at = Column(String)


class ModelProfile(Base):
    __tablename__ = "model_profile"

    id = Column(Integer, primary_key=True, autoincrement=True)
    name = Column(String, nullable=False, unique=True)
```

```
    family = Column(String, nullable=False, default="custom")
    code_path = Column(String, nullable=False, default="")
    weights_path = Column(String, nullable=False, default="")
    entry = Column(String, nullable=False, default="")
    default_config_json = Column(Text, nullable=False, default="{}")
    needs_gpu = Column(Integer, nullable=False, default=1)
    allowed_inputs = Column(String, nullable=False, default="rgb,depth,pose,action")
    status = Column(String, nullable=False, default="unknown")
    description = Column(Text, nullable=False, default="")
    created_at = Column(String)
    updated_at = Column(String)


class EvalTask(Base):
    __tablename__ = "eval_task"

    id = Column(Integer, primary_key=True, autoincrement=True)
    name = Column(String, nullable=False)
    dataset_id = Column(Integer, ForeignKey("dataset.id", ondelete="RESTRICT"), nullable=False)
    split_id = Column(Integer, ForeignKey("split.id", ondelete="RESTRICT"), nullable=False)
    model_profile_id = Column(
        Integer, ForeignKey("model_profile.id", ondelete="RESTRICT"), nullable=False
    )
    config_json = Column(Text, nullable=False, default="{}")
    status = Column(String, nullable=False, default="queued")
    priority = Column(Integer, nullable=False, default=5)
    progress_done = Column(Integer, nullable=False, default=0)
    progress_total = Column(Integer, nullable=False, default=0)
    metrics_json = Column(Text, nullable=False, default="{}")
    error_msg = Column(Text, nullable=False, default="")
    gpu_ids = Column(String, nullable=False, default="")
    pid = Column(Integer)
    created_by = Column(Integer, ForeignKey("app_user.id", ondelete="SET NULL"))
    queued_at = Column(String)
    started_at = Column(String)
    finished_at = Column(String)
    duration_s = Column(Float)
    note = Column(Text, nullable=False, default="")


class TaskEpisode(Base):
    __tablename__ = "task_episode"

    id = Column(Integer, primary_key=True, autoincrement=True)
    task_id = Column(Integer, ForeignKey("eval_task.id", ondelete="CASCADE"), nullable=False)
    episode_id = Column(Integer, ForeignKey("episode.id", ondelete="CASCADE"), nullable=False)
    status = Column(String, nullable=False, default="pending")
    success = Column(Integer, nullable=False, default=0)
    spl = Column(Float, nullable=False, default=0)
    ne = Column(Float, nullable=False, default=0)
```

```
    os = Column(Float, nullable=False, default=0)
    ndtw = Column(Float, nullable=False, default=0)
    tl = Column(Float, nullable=False, default=0)
    steps = Column(Integer, nullable=False, default=0)
    latency_avg_ms = Column(Float, nullable=False, default=0)
    latency_p95_ms = Column(Float, nullable=False, default=0)
    gpu_mem_peak_mb = Column(Integer, nullable=False, default=0)
    tokens_peak = Column(Integer, nullable=False, default=0)
    ctx_len_peak = Column(Integer, nullable=False, default=0)
    failure_reason = Column(String, nullable=False, default="")
    manual_reason = Column(String, nullable=False, default="")
    traj_path = Column(String, nullable=False, default="")
    log_path = Column(String, nullable=False, default="")
    updated_at = Column(String)


class ResourceSample(Base):
    __tablename__ = "resource_sample"

    id = Column(Integer, primary_key=True, autoincrement=True)
    task_id = Column(Integer, ForeignKey("eval_task.id", ondelete="CASCADE"), nullable=False)
    ts = Column(String)
    step = Column(Integer, nullable=False, default=0)
    gpu_mem_mb = Column(Integer, nullable=False, default=0)
    latency_ms = Column(Float, nullable=False, default=0)
    tokens = Column(Integer, nullable=False, default=0)
    ctx_len = Column(Integer, nullable=False, default=0)


class Artifact(Base):
    __tablename__ = "artifact"

    id = Column(Integer, primary_key=True, autoincrement=True)
    task_id = Column(Integer, ForeignKey("eval_task.id", ondelete="CASCADE"), nullable=False)
    episode_id = Column(Integer, ForeignKey("task_episode.id", ondelete="CASCADE"))
    kind = Column(String, nullable=False, default="trajectory")
    name = Column(String, nullable=False, default="")
    path = Column(String, nullable=False)
    size_bytes = Column(Integer, nullable=False, default=0)
    sha256 = Column(String, nullable=False, default="")
    created_at = Column(String)
"""数据集、Episode 与切分映射。"""

from __future__ import annotations

from sqlalchemy import Column, Float, ForeignKey, Integer, String, Text

from ..db.base import Base
```

```
class Dataset(Base):
    __tablename__ = "dataset"

    id = Column(Integer, primary_key=True, autoincrement=True)
    name = Column(String, nullable=False)
    version = Column(String, nullable=False, default="")
    kind = Column(String, nullable=False, default="r2r_ce")
    root_path = Column(String, nullable=False)
    instruction_lang = Column(String, nullable=False, default="en")
    scene_count = Column(Integer, nullable=False, default=0)
    episode_count = Column(Integer, nullable=False, default=0)
    status = Column(String, nullable=False, default="unknown")
    last_scan_at = Column(String)
    meta_json = Column(Text, nullable=False, default="{}")
    created_at = Column(String)
    updated_at = Column(String)


class Episode(Base):
    __tablename__ = "episode"

    id = Column(Integer, primary_key=True, autoincrement=True)
    dataset_id = Column(Integer, ForeignKey("dataset.id", ondelete="CASCADE"), nullable=False)
    episode_id = Column(String, nullable=False)
    scene_id = Column(String, nullable=False, default="")
    instruction = Column(Text, nullable=False, default="")
    instruction_len = Column(Integer, nullable=False, default=0)
    traj_len = Column(Float, nullable=False, default=0)
    duration_s = Column(Float, nullable=False, default=0)
    start_pose_json = Column(Text, nullable=False, default="[]")
    goal_pose_json = Column(Text, nullable=False, default="[]")
    has_depth = Column(Integer, nullable=False, default=0)
    meta_json = Column(Text, nullable=False, default="{}")


class Split(Base):
    __tablename__ = "split"

    id = Column(Integer, primary_key=True, autoincrement=True)
    dataset_id = Column(Integer, ForeignKey("dataset.id", ondelete="CASCADE"), nullable=False)
    name = Column(String, nullable=False)
    kind = Column(String, nullable=False, default="subset")
    base_split = Column(String, nullable=False, default="val_unseen")
    seed = Column(Integer, nullable=False, default=42)
    episode_count = Column(Integer, nullable=False, default=0)
    filter_json = Column(Text, nullable=False, default="{}")
    created_at = Column(String)


class SplitEpisode(Base):
```

```
    __tablename__ = "split_episode"

    split_id = Column(
        Integer, ForeignKey("split.id", ondelete="CASCADE"), primary_key=True, nullable=False
    )
    episode_id = Column(
        Integer, ForeignKey("episode.id", ondelete="CASCADE"), primary_key=True, nullable=False
    )
    ordinal = Column(Integer, nullable=False, default=0)
"""用户与审计日志映射。"""

from __future__ import annotations

from sqlalchemy import Column, ForeignKey, Integer, String, Text

from ..db.base import Base


class AppUser(Base):
    __tablename__ = "app_user"

    id = Column(Integer, primary_key=True, autoincrement=True)
    username = Column(String, nullable=False, unique=True)
    display_name = Column(String, nullable=False, default="")
    password_hash = Column(String, nullable=False)
    password_salt = Column(String, nullable=False)
    role = Column(String, nullable=False, default="operator")
    enabled = Column(Integer, nullable=False, default=1)
    last_login_at = Column(String)
    created_at = Column(String)

    @property
    def is_admin(self) -> bool:
        return self.role == "admin"


class AuditLog(Base):
    __tablename__ = "audit_log"

    id = Column(Integer, primary_key=True, autoincrement=True)
    user_id = Column(Integer, ForeignKey("app_user.id", ondelete="SET NULL"))
    username = Column(String, nullable=False, default="")
    action = Column(String, nullable=False)
    target_type = Column(String, nullable=False, default="")
    target_id = Column(String, nullable=False, default="")
    detail_json = Column(Text, nullable=False, default="{}")
    client_ip = Column(String, nullable=False, default="")
    created_at = Column(String)
"""备份库导入来源账本与资料索引映射。"""
```

```
from __future__ import annotations

from sqlalchemy import Column, Integer, String, Text

from ..db.base import Base


class BackupAsset(Base):
    __tablename__ = "backup_asset"

    id = Column(Integer, primary_key=True, autoincrement=True)
    source_key = Column(String, nullable=False, unique=True)
    root_path = Column(String, nullable=False, default="")
    rel_path = Column(String, nullable=False, default="")
    section = Column(String, nullable=False, default="assets")
    kind = Column(String, nullable=False, default="file")
    name = Column(String, nullable=False, default="")
    size_bytes = Column(Integer, nullable=False, default=0)
    mtime = Column(String, nullable=False, default="")
    signature = Column(String, nullable=False, default="")
    entity_type = Column(String, nullable=False, default="")
    entity_id = Column(Integer)
    status = Column(String, nullable=False, default="new")
    message = Column(Text, nullable=False, default="")
    meta_json = Column(Text, nullable=False, default="{}")
    imported_at = Column(String)
    updated_at = Column(String)
"""适配器协议与运行上下文。

适配器只负责"跑一条 episode 并产出标准记录", 不写数据库;
调度器负责持久化、事件推送与状态迁移, 便于替换推理后端。
"""

from __future__ import annotations

import threading
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any, Callable, Iterator


@dataclass
class EpisodeRecord:
    episode_key: str
    scene_id: str = ""
    success: bool = False
    spl: float = 0.0
    ne: float = 0.0
    os: float = 0.0
    ndtw: float = 0.0
```

```
    tl: float = 0.0
    steps: int = 0
    latencies: list[float] = field(default_factory=list)
    gpu_mem_peak_mb: int = 0
    tokens_peak: int = 0
    ctx_len_peak: int = 0
    failure_reason: str = ""
    frames: list[dict[str, Any]] = field(default_factory=list)

    @property
    def latency_avg_ms(self) -> float:
        if not self.latencies:
            return 0.0
        return round(sum(self.latencies) / len(self.latencies), 2)

    @property
    def latency_p95_ms(self) -> float:
        if not self.latencies:
            return 0.0
        ordered = sorted(self.latencies)
        index = min(len(ordered) - 1, int(len(ordered) * 0.95))
        return round(ordered[index], 2)


@dataclass
class RunContext:
    task_id: int
    task_name: str
    run_dir: Path
    config: dict[str, Any]
    episodes: list[dict[str, Any]]
    model: dict[str, Any]
    dataset: dict[str, Any]
    log: Callable[[str], None]
    progress: Callable[[int, int, str], None]
    sample: Callable[[dict[str, Any]], None]
    cancel_event: threading.Event = field(default_factory=threading.Event)
    pause_event: threading.Event = field(default_factory=threading.Event)
    stopped: bool = False

    @property
    def frames_dir(self) -> Path:
        directory = self.run_dir / "frames"
        directory.mkdir(parents=True, exist_ok=True)
        return directory

    def should_cancel(self) -> bool:
        return self.cancel_event.is_set()

    def wait_if_paused(self) -> None:
```

```
        while self.pause_event.is_set() and not self.cancel_event.is_set():
            threading.Event().wait(0.3)


class BaseAdapter:
    name = "base"
    allowed_inputs: set[str] = {"rgb"}

    def __init__(self, ctx: RunContext):
        self.ctx = ctx

    def prepare(self) -> None:
        """校验运行条件，失败时抛出 RuntimeError 并附带可读原因。"""

    def run(self) -> Iterator[EpisodeRecord]:
        raise NotImplementedError

    def stop(self) -> None:
        """终止底层进程（取消时调用），默认无操作。"""

    def finish(self) -> dict[str, Any]:
        return {}


def build_adapter(ctx: RunContext) -> BaseAdapter:
    family = str(ctx.model.get("family", "")).lower()
    if family == "streamvln":
        return StreamVLNAdapter(ctx)
    if family == "mock":
        return MockAdapter(ctx)
    entry = str(ctx.model.get("entry") or "")
    if entry.endswith("MockAdapter"):
        return MockAdapter(ctx)
    raise RuntimeError(f"暂不支持的模型家族：{family or '未配置'}")


# 延迟导入，避免循环引用
from .mock_adapter import MockAdapter  # noqa: E402
from .streamvln_adapter import StreamVLNAdapter  # noqa: E402
"""StreamVLN 真实推理适配器。

调用方式与上游 `scripts/run_eval_kaggle.sh` 一致：
    torchrun --nproc_per_node=1 --master_port=<port> <code>/streamvln/streamvln_eval.py \
        --model_path <weights> --eval_split <split> --output_path <raw> --seed <seed> [extra]

运行期持续 tail `<raw>/result.json`（JSONL，一行一条 episode），
解析为标准记录后交给调度器，因此进度是逐条推进的，而不是等进程结束。
"""

from __future__ import annotations
```

```
import json
import os
import shutil
import signal
import subprocess
import threading
import time
from collections import deque
from pathlib import Path
from queue import Empty, Queue
from typing import Any, Iterator

from .base import BaseAdapter, EpisodeRecord

# 回放工作台一次最多展示的帧数（与 replay_service.MAX_FRAMES 对齐）
MAX_REPLAY_FRAMES = 600


def _as_float(value: Any, default: float = 0.0) -> float:
    try:
        return float(value)
    except (TypeError, ValueError):
        return default


def _as_int(value: Any, default: int = 0) -> int:
    try:
        return int(float(value))
    except (TypeError, ValueError):
        return default


def _supports_frame_dump(script: Path) -> bool:
    """上游脚本是否支持 --dump_frames（老版本代码里没有这个开关）。"""
    try:
        return "--dump_frames" in script.read_text(encoding="utf-8", errors="replace")
    except OSError:
        return False


class StreamVLNAdapter(BaseAdapter):
    name = "streamvln"

    def __init__(self, ctx):
        super().__init__(ctx)
        self.proc: subprocess.Popen | None = None
        self.result_path: Path | None = None
        # 逐帧回放元数据目录（评测脚本 --dump_frames 的输出），脚本不支持时为 None
        self.replay_dir: Path | None = None
```

```
        self._buffer = ""
        self._offset = 0
        self._seen: set[str] = set()
        self._log_queue: Queue[str] = Queue()
        self._tail: deque[str] = deque(maxlen=40)
        self._reader: threading.Thread | None = None

    # ---------------------------------------------------------------- 准备

    def prepare(self) -> None:
        model = self.ctx.model
        code_path = Path(str(model.get("code_path") or "")).expanduser()
        if not code_path.exists():
            raise RuntimeError(f"代码目录不存在: {code_path}")

        script = code_path / "streamvln" / "streamvln_eval.py"
        if not script.exists():
            raise RuntimeError(f"未找到评测脚本: {script}")

        weights = Path(str(model.get("weights_path") or "")).expanduser()
        if not weights.exists():
            raise RuntimeError(f"权重目录不存在: {weights}")

        launcher = str(model.get("entry") or "torchrun").strip()
        launcher_path = launcher if Path(launcher).is_absolute() else shutil.which(launcher)
        if not launcher_path or not Path(launcher_path).exists():
            raise RuntimeError(f"找不到启动器: {launcher}, 请检查模型档案中的入口配置")

        raw_dir = self.ctx.run_dir / "raw"
        raw_dir.mkdir(parents=True, exist_ok=True)
        self.result_path = raw_dir / "result.json"

        # 逐帧回放: 上游脚本带 --dump_frames 才开启, 老版本代码自动降级为「只有汇总指标」
        self.replay_dir = raw_dir / "frames" if _supports_frame_dump(script) else None
        if self.replay_dir is None:
            self.ctx.log("评测脚本不支持 --dump_frames, 本次不产出逐帧回放数据")

        command = self._build_command(str(launcher_path), script, weights, raw_dir)
        self.ctx.log("启动命令: " + " ".join(command))

        env = os.environ.copy()
        config = self.ctx.config
        gpu_ids = str(config.get("gpu_ids") or "").strip()
        env["CUDA_VISIBLE_DEVICES"] = gpu_ids or "0"
        env.setdefault("PROTOCOL_BUFFERS_PYTHON_IMPLEMENTATION", "python")
        env.setdefault("PYTORCH_CUDA_ALLOC_CONF", "expandable_segments:True")
        env.setdefault("MAGNUM_LOG", "quiet")
        env.setdefault("HABITAT_SIM_LOG", "quiet")

        self.proc = subprocess.Popen(  # noqa: S603 - 命令由本地配置拼装
```

```
            command,
            cwd=str(code_path),
            stdout=subprocess.PIPE,
            stderr=subprocess.STDOUT,
            text=True,
            bufsize=1,
            env=env,
            start_new_session=True,
        )
        self._reader = threading.Thread(target=self._read_output, daemon=True)
        self._reader.start()
        self.ctx.log(f"子进程已启动 pid={self.proc.pid}")

    def _build_command(
        self, launcher: str, script: Path, weights: Path, raw_dir: Path
    ) -> list[str]:
        config = self.ctx.config
        eval_split = str(config.get("eval_split") or "val_unseen")
        seed = int(config.get("seed", 42))
        command = [
            launcher,
            "--nproc_per_node=1",
            f"--master_port={config.get('master_port', 29611)}",
            str(script),
            "--model_path",
            str(weights),
            "--eval_split",
            eval_split,
            "--output_path",
            str(raw_dir),
            "--seed",
            str(seed),
        ]
        if self.replay_dir is not None:
            command.extend(["--dump_frames", str(self.replay_dir)])

        extra = config.get("adapter", {}).get("extra_args") or []
        if extra:
            command.extend([str(item) for item in extra])
            return command

        command.extend(["--num_history", str(int(config.get("history_len", 8)))])
        command.extend(["--num_future_steps", str(int(config.get("num_future_steps", 4)))])
        if config.get("memory_policy") == "spatial_prune":
            prune_ratio = float(config.get("prune_ratio", 0.0))
            command.extend(["--prune_mode", "voxel"])
            if 0 < prune_ratio < 1:
                command.extend(["--prune_keep_ratio", f"{1 - prune_ratio:.3f}"])
        return command
```

```
1651    def _read_output(self) -> None:
1652        assert self.proc is not None and self.proc.stdout is not None
1653        for line in self.proc.stdout:
1654            text = line.rstrip()
1655            if text:
1656                self._tail.append(text)
1657                self._log_queue.put(text)

1659    # ---------------------------------------------------------------- 执行

1661    def run(self) -> Iterator[EpisodeRecord]:
1662        if self.proc is None:
1663            self.prepare()
1664        assert self.proc is not None

1666        idle_rounds = 0
1667        while True:
1668            if self.ctx.should_cancel():
1669                self.stop()
1670                return

1672            self._flush_logs()

1674            produced = False
1675            for payload in self._read_new_records():
1676                record = self._to_record(payload)
1677                if record is None or record.episode_key in self._seen:
1678                    continue
1679                self._seen.add(record.episode_key)
1680                produced = True
1681                yield record

1683            if self.proc.poll() is not None:
1684                idle_rounds += 1
1685                if not produced and idle_rounds >= 2:
1686                    break
1687            else:
1688                idle_rounds = 0

1690            time.sleep(0.6)

1692        self._flush_logs()

1694        if self.proc.returncode not in (0, None):
1695            tail = "\n".join(list(self._tail)[-12:])
1696            if "out of memory" in tail.lower():
1697                raise RuntimeError("推理进程显存不足 (OOM) \n" + tail)
1698            raise RuntimeError(f"推理进程退出码 {self.proc.returncode}\n{tail}")

1700    def _flush_logs(self) -> None:
```

```
        while True:
            try:
                line = self._log_queue.get_nowait()
            except Empty:
                return
            self.ctx.log(line)

    def _read_new_records(self) -> list[dict[str, Any]]:
        """增量读取结果文件，只解析新增的完整行。"""
        if self.result_path is None or not self.result_path.exists():
            return []

        with self.result_path.open("rb") as handle:
            handle.seek(self._offset)
            data = handle.read()
        if not data:
            return []
        self._offset += len(data)

        text = self._buffer + data.decode("utf-8", errors="replace")
        lines = text.split("\n")
        self._buffer = lines.pop()  # 最后一段可能不完整，留到下次拼接

        payloads: list[dict[str, Any]] = []
        for line in lines:
            line = line.strip()
            if not line:
                continue
            try:
                payloads.append(json.loads(line))
            except json.JSONDecodeError:
                continue
        return payloads

    def _load_frames(self, scene_id: str, episode_key: str) -> list[dict[str, Any]]:
        """读取 `--dump_frames` 产出的逐帧元数据，形状与回放服务一致。"""
        if self.replay_dir is None or not self.replay_dir.is_dir():
            return []

        target = self.replay_dir / f"{scene_id}_{episode_key}.jsonl"
        if not target.is_file():
            # 兼容只按 episode_id 命名的实现
            matches = sorted(self.replay_dir.glob(f"*_{episode_key}.jsonl"))
            if not matches:
                return []
            target = matches[0]

        try:
            text = target.read_text(encoding="utf-8", errors="replace")
        except OSError:
```

```
            return []

        frames: list[dict[str, Any]] = []
        for line in text.splitlines():
            line = line.strip()
            if not line:
                continue
            try:
                raw = json.loads(line)
            except json.JSONDecodeError:
                continue
            if not isinstance(raw, dict):
                continue
            pose = raw.get("pose")
            if not isinstance(pose, (list, tuple)) or len(pose) < 3:
                continue
            try:
                point = [round(float(value), 3) for value in pose[:3]]
            except (TypeError, ValueError):
                continue

            before = _as_int(raw.get("tokens_before_prune"))
            after = _as_int(raw.get("tokens_after_prune"), before)
            frames.append(
                {
                    "idx": _as_int(raw.get("idx"), len(frames)),
                    "t": round(_as_float(raw.get("t")), 3),
                    "pose": point,
                    "yaw": round(_as_float(raw.get("yaw")), 4),
                    "action": str(raw.get("action") or "forward"),
                    "latency_ms": round(_as_float(raw.get("latency_ms")), 2),
                    "window_len": _as_int(raw.get("window_len")),
                    "slow_update": bool(raw.get("slow_update")),
                    "tokens_before_prune": before,
                    "tokens_after_prune": after,
                    "ctx_len": _as_int(raw.get("ctx_len"), after),
                }
            )

        frames.sort(key=lambda item: item["idx"])
        return frames[:MAX_REPLAY_FRAMES]

    def _to_record(self, payload: dict[str, Any]) -> EpisodeRecord | None:
        episode_key = payload.get("episode_id")
        if episode_key is None:
            return None

        key = str(episode_key)
        scene_id = str(payload.get("scene_id") or "")
        steps = int(payload.get("steps") or 0)
```

```
        elapsed = float(payload.get("elapsed_s") or 0.0)
        success = bool(payload.get("success"))

        frames = self._load_frames(scene_id, key)
        step_latencies = [frame["latency_ms"] for frame in frames if frame["latency_ms"] > 0]
        if step_latencies:
            latencies = step_latencies
        else:
            average = round(elapsed / steps * 1000, 2) if steps else 0.0
            latencies = [average] if average else []

        # 逐帧有真实记账就用逐帧峰值；没有则退回 episode 级均值，绝不放大伪造
        tokens_peak = max((frame["tokens_before_prune"] for frame in frames), default=0)
        if not tokens_peak:
            tokens_peak = int(_as_float(payload.get("mean_memory_tokens_before")))
        ctx_len_peak = max((frame["ctx_len"] for frame in frames), default=0)
        if not ctx_len_peak:
            ctx_len_peak = int(_as_float(payload.get("mean_prompt_tokens"))) or int(
                _as_float(payload.get("mean_memory_tokens_after"))
            )

        reason = ""
        if not success:
            max_steps = int(self.ctx.config.get("max_steps", 500))
            reason = "timeout" if steps >= max_steps else "early_stop"

        return EpisodeRecord(
            episode_key=key,
            scene_id=scene_id,
            success=success,
            spl=float(payload.get("spl") or 0.0),
            ne=float(payload.get("ne") or 0.0),
            os=float(payload.get("os") or 0.0),
            ndtw=float(payload.get("ndtw") or 0.0),
            tl=float(payload.get("tl") or 0.0),
            steps=steps,
            latencies=latencies,
            gpu_mem_peak_mb=int(_as_float(payload.get("peak_gpu_mem_mb"))),
            tokens_peak=tokens_peak,
            ctx_len_peak=ctx_len_peak,
            failure_reason=reason,
            frames=frames,
        )

    def stop(self) -> None:
        if self.proc is None or self.proc.poll() is not None:
            return
        try:
            os.killpg(os.getpgid(self.proc.pid), signal.SIGTERM)
        except (ProcessLookupError, PermissionError):
```

```
            return
        try:
            self.proc.wait(timeout=10)
        except subprocess.TimeoutExpired:
            try:
                os.killpg(os.getpgid(self.proc.pid), signal.SIGKILL)
            except (ProcessLookupError, PermissionError):
                pass
"""评测任务调度器。

单进程内的异步调度：队列 + 并发额度 + GPU 互斥，任务体在独立线程中执行，
通过事件总线向 SSE 推送进度、资源采样与日志。每条 episode 完成即落库，
异常退出后可按已完成进度继续。
"""

from __future__ import annotations

import asyncio
import json
import logging
import time
from datetime import datetime
from pathlib import Path
from typing import Any

from sqlalchemy import select, update

from ..adapters import BaseAdapter, EpisodeRecord, RunContext, build_adapter
from ..core.config import Settings, get_settings
from ..core.events import BUS
from ..db.session import session_scope
from ..models import (
    Dataset,
    Episode,
    EvalTask,
    ModelProfile,
    ResourceSample,
    TaskEpisode,
)
from . import model_service, task_service

logger = logging.getLogger(__name__)


class Scheduler:
    def __init__(self, settings: Settings | None = None) -> None:
        self.settings = settings or get_settings()
        self._queue: asyncio.Queue[int] = asyncio.Queue()
        self._semaphore = asyncio.Semaphore(max(1, self.settings.scheduler.max_parallel_tasks))
        self._gpu_locks: dict[str, asyncio.Lock] = {}
```

```
        self._runtime: dict[int, dict[str, Any]] = {}
        self._dispatcher: asyncio.Task | None = None
        self._loop: asyncio.AbstractEventLoop | None = None
        self._stopping = False

    # ---------------------------------------------------------------- 生命周期

    async def start(self) -> None:
        self._loop = asyncio.get_running_loop()
        BUS.bind_loop(self._loop)

        with session_scope() as db:
            stale = task_service.mark_stale_tasks(db)
            queued = [
                task.id
                for task in db.scalars(
                    select(EvalTask).where(EvalTask.status == "queued").order_by(EvalTask.id)
                ).all()
            ]
        if stale:
            logger.warning("已将 %s 个中断任务标记为失败", stale)

        for task_id in queued:
            self._queue.put_nowait(task_id)
        if queued:
            logger.info("重新入队 %s 个待执行任务", len(queued))

        self._dispatcher = asyncio.create_task(self._dispatch())

    async def shutdown(self) -> None:
        self._stopping = True
        if self._dispatcher:
            self._dispatcher.cancel()
        for task_id, runtime in list(self._runtime.items()):
            runtime["ctx"].cancel_event.set()
            adapter = runtime.get("adapter")
            if adapter is not None:
                await asyncio.to_thread(adapter.stop)
            self._emit(task_id, "state", {"status": "canceled", "reason": "shutdown"})

    # ---------------------------------------------------------------- 入队

    def submit(self, task_id: int) -> None:
        """任意线程安全地入队。"""
        if self._loop and self._loop.is_running():
            self._loop.call_soon_threadsafe(self._queue.put_nowait, task_id)
        else:
            self._queue.put_nowait(task_id)

    async def _dispatch(self) -> None:
```

```
        while not self._stopping:
            task_id = await self._queue.get()
            await self._semaphore.acquire()
            asyncio.create_task(self._run_wrapper(task_id))

    async def _run_wrapper(self, task_id: int) -> None:
        gpu_key = ""
        try:
            with session_scope() as db:
                task = db.get(EvalTask, task_id)
                gpu_key = (task.gpu_ids or "").split(",")[0].strip() if task else ""
        except Exception:  # noqa: BLE001 - 读取失败时按无 GPU 约束处理
            logger.exception("读取任务 GPU 配置失败 task=%s", task_id)

        lock = None
        if gpu_key:
            lock = self._gpu_locks.setdefault(gpu_key, asyncio.Lock())
            await lock.acquire()
        try:
            await asyncio.to_thread(self._run_task, task_id)
        except Exception:  # noqa: BLE001
            logger.exception("任务执行线程异常 task=%s", task_id)
        finally:
            if lock is not None:
                lock.release()
            self._semaphore.release()

    # ---------------------------------------------------------------- 执行

    def _run_task(self, task_id: int) -> None:
        with session_scope() as db:
            task = db.get(EvalTask, task_id)
            if task is None or task.status not in ("queued", "paused"):
                return

            task.status = "running"
            task.started_at = task.started_at or task_service.now_str()
            task.error_msg = ""
            config = json.loads(task.config_json or "{}")

            rows = db.execute(
                select(TaskEpisode, Episode)
                .join(Episode, Episode.id == TaskEpisode.episode_id)
                .where(TaskEpisode.task_id == task.id, TaskEpisode.status == "pending")
                .order_by(Episode.id)
            ).all()

            episodes = [
                {
                    "episode_key": episode.episode_id,
```

```
                "scene_id": episode.scene_id,
                "instruction": episode.instruction,
                "traj_len": episode.traj_len,
                "start_pose": json.loads(episode.start_pose_json or "[]"),
            }
            for _, episode in rows
        ]
        row_ids = {episode.episode_id: row.id for row, episode in rows}

        dataset = db.get(Dataset, task.dataset_id)
        model = db.get(ModelProfile, task.model_profile_id)
        dataset_info = {
            "id": dataset.id,
            "name": dataset.name,
            "root_path": dataset.root_path,
            "kind": dataset.kind,
        }
        model_info = model_service.to_dict(model) if model else {}

    if not episodes:
        self._finalize(task_id, error="没有待执行的 episode")
        return

    run_dir = task_service.task_run_dir(task_id)
    (run_dir / "config.json").write_text(
        json.dumps(config, ensure_ascii=False, indent=2), encoding="utf-8"
    )
    log_path = run_dir / "run.log"

    def append_log(text: str) -> None:
        stamp = datetime.now().strftime("%H:%M:%S")
        line = f"[{stamp}] {text}"
        with log_path.open("a", encoding="utf-8") as handle:
            handle.write(line + "\n")
        self._emit(task_id, "log", {"level": "info", "text": text})

    ctx = RunContext(
        task_id=task_id,
        task_name="",
        run_dir=run_dir,
        config=config,
        episodes=episodes,
        model=model_info,
        dataset=dataset_info,
        log=append_log,
        progress=lambda done, total, key: None,
        sample=lambda payload: None,
    )

    self._runtime[task_id] = {"ctx": ctx, "adapter": None}
```

```
        self._emit(task_id, "state", {"status": "running", "total": len(episodes)})
        append_log(f"开始执行，待评测 {len(episodes)} 条 episode")

        adapter: BaseAdapter | None = None
        processed = 0
        started = time.time()
        error: str | None = None

        try:
            adapter = build_adapter(ctx)
            self._runtime[task_id]["adapter"] = adapter
            adapter.prepare()

            allowed_keys = set(row_ids)
            for record in adapter.run():
                if ctx.should_cancel():
                    break

                if record.episode_key not in allowed_keys:
                    append_log(f"忽略切分外的结果：{record.episode_key}")
                    continue

                self._persist(task_id, row_ids[record.episode_key], record, run_dir, ctx)
                processed += 1
                self._emit(
                    task_id,
                    "progress",
                    {
                        "done": processed,
                        "total": len(episodes),
                        "episode": record.episode_key,
                        "success": record.success,
                        "spl": record.spl,
                    },
                )
                self._emit(
                    task_id,
                    "sample",
                    {
                        "step": processed,
                        "gpu_mem_mb": record.gpu_mem_peak_mb,
                        "latency_ms": record.latency_avg_ms,
                        "tokens": record.tokens_peak,
                        "ctx_len": record.ctx_len_peak,
                    },
                )

                if ctx.pause_event.is_set():
                    self._set_status(task_id, "paused")
                    self._emit(task_id, "state", {"status": "paused"})
```

```
                    append_log("任务已暂停，等待恢复")
                    while ctx.pause_event.is_set() and not ctx.should_cancel():
                        time.sleep(0.3)
                    if ctx.should_cancel():
                        break
                    self._set_status(task_id, "running")
                    self._emit(task_id, "state", {"status": "running"})
                    append_log("任务已恢复")

            adapter.finish()
        except Exception as exc:  # noqa: BLE001 - 需要把失败原因回传界面
            error = str(exc)
            logger.exception("任务执行失败 task=%s", task_id)
            append_log(f"执行失败：{error}")
        finally:
            if ctx.should_cancel() and adapter is not None:
                try:
                    adapter.stop()
                except Exception:  # noqa: BLE001 - 清理阶段忽略
                    logger.exception("终止适配器失败 task=%s", task_id)
            self._runtime.pop(task_id, None)

        if ctx.should_cancel():
            append_log("任务已取消")
            self._finalize(task_id, canceled=True)
        elif error:
            self._finalize(task_id, error=error)
        else:
            append_log(f"执行完成，共处理 {processed} 条，用时 {time.time() - started:.1f}s")
            self._finalize(task_id)

    def _persist(
        self,
        task_id: int,
        row_id: int,
        record: EpisodeRecord,
        run_dir: Path,
        ctx: RunContext,
    ) -> None:
        frames_path = ""
        if record.frames:
            directory = run_dir / "frames"
            directory.mkdir(parents=True, exist_ok=True)
            target = directory / f"{record.episode_key}.jsonl"
            with target.open("w", encoding="utf-8") as handle:
                for frame in record.frames:
                    handle.write(json.dumps(frame, ensure_ascii=False) + "\n")
            frames_path = str(target)

        with session_scope() as db:
```

```
            db.execute(
                update(TaskEpisode)
                .where(TaskEpisode.id == row_id)
                .values(
                    status="success" if record.success else "failed",
                    success=1 if record.success else 0,
                    spl=record.spl,
                    ne=record.ne,
                    os=record.os,
                    ndtw=record.ndtw,
                    tl=record.tl,
                    steps=record.steps,
                    latency_avg_ms=record.latency_avg_ms,
                    latency_p95_ms=record.latency_p95_ms,
                    gpu_mem_peak_mb=record.gpu_mem_peak_mb,
                    tokens_peak=record.tokens_peak,
                    ctx_len_peak=record.ctx_len_peak,
                    failure_reason=record.failure_reason,
                    traj_path=frames_path,
                    updated_at=task_service.now_str(),
                )
            )
            task = db.get(EvalTask, task_id)
            progress = 0
            if task is not None:
                task.progress_done = (task.progress_done or 0) + 1
                progress = task.progress_done

            db.add(
                ResourceSample(
                    task_id=task_id,
                    ts=task_service.now_str(),
                    step=progress,
                    gpu_mem_mb=record.gpu_mem_peak_mb,
                    latency_ms=record.latency_avg_ms,
                    tokens=record.tokens_peak,
                    ctx_len=record.ctx_len_peak,
                )
            )

    def _finalize(self, task_id: int, error: str | None = None, canceled: bool = False) -> None:
        with session_scope() as db:
            task = db.get(EvalTask, task_id)
            if task is None:
                return
            metrics = task_service.aggregate_metrics(db, task)
            if canceled:
                task.status = "canceled"
            elif error:
                task.status = "failed"
```

```
                task.error_msg = error[:1000]
            else:
                task.status = "succeeded"
            task.finished_at = task_service.now_str()
            if task.started_at:
                try:
                    start = datetime.strptime(task.started_at, "%Y-%m-%d %H:%M:%S")
                    end = datetime.strptime(task.finished_at, "%Y-%m-%d %H:%M:%S")
                    task.duration_s = round((end - start).total_seconds(), 1)
                except ValueError:
                    task.duration_s = None

        self._emit(task_id, "state", {"status": task.status if task else ""})
        self._emit(task_id, "done", {"status": task.status if task else "", "metrics": metrics})
        logger.info("任务结束 task=%s status=%s metrics=%s", task_id, task.status if task else "-", metrics)

    def _set_status(self, task_id: int, status: str) -> None:
        with session_scope() as db:
            task = db.get(EvalTask, task_id)
            if task is not None:
                task.status = status

    @staticmethod
    def _emit(task_id: int, event: str, data: dict[str, Any]) -> None:
        BUS.publish_threadsafe(task_id, event, data)

    # ---------------------------------------------------------------- 控制

    def set_max_parallel(self, value: int) -> None:
        """运行期调整并发额度，新提交的任务立即使用新的信号量。"""
        limit = max(1, min(int(value), 8))
        self.settings.scheduler.max_parallel_tasks = limit
        self._semaphore = asyncio.Semaphore(limit)
        logger.info("并发额度已调整为 %s", limit)

    async def pause(self, task_id: int) -> str:
        runtime = self._runtime.get(task_id)
        if runtime is None:
            raise RuntimeError("任务当前未在运行，无法暂停")
        runtime["ctx"].pause_event.set()
        return "pausing"

    async def resume(self, task_id: int) -> str:
        runtime = self._runtime.get(task_id)
        if runtime is None:
            with session_scope() as db:
                task = task_service.get_task(db, task_id)
                if task.status != "paused":
                    raise RuntimeError("任务不在暂停状态")
                task.status = "queued"
```

```
            self.submit(task_id)
            return "queued"
        runtime["ctx"].pause_event.clear()
        return "running"

    async def cancel(self, task_id: int) -> str:
        with session_scope() as db:
            task = task_service.get_task(db, task_id)
            if task.status in task_service.TERMINAL_STATUS:
                return task.status

        runtime = self._runtime.get(task_id)
        if runtime is None:
            with session_scope() as db:
                task = task_service.get_task(db, task_id)
                task.status = "canceled"
                task.finished_at = task_service.now_str()
            self._emit(task_id, "state", {"status": "canceled"})
            return "canceled"

        runtime["ctx"].cancel_event.set()
        adapter = runtime.get("adapter")
        if adapter is not None:
            await asyncio.to_thread(adapter.stop)
        return "canceling"


_scheduler: Scheduler | None = None


def get_scheduler() -> Scheduler:
    global _scheduler
    if _scheduler is None:
        _scheduler = Scheduler()
    return _scheduler
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import ElementPlus from 'element-plus'
import zhCn from 'element-plus/es/locale/lang/zh-cn'
import * as ElementPlusIconsVue from '@element-plus/icons-vue'
import 'element-plus/dist/index.css'

import App from './App.vue'
import router from './router'
import './styles/theme.css'

const app = createApp(App)

for (const [name, component] of Object.entries(ElementPlusIconsVue)) {
  app.component(name, component)
```

```
}

app.use(createPinia())
app.use(router)
app.use(ElementPlus, { locale: zhCn })
app.mount('#app')
<template>
  <router-view />
</template>

<script setup>
</script>
import { createRouter, createWebHistory } from 'vue-router'
import { useAuthStore } from '@/store/auth'

const routes = [
  {
    path: '/login',
    name: 'login',
    component: () => import('@/views/Login.vue'),
    meta: { public: true, title: '登录' },
  },
  {
    path: '/',
    component: () => import('@/layouts/DefaultLayout.vue'),
    redirect: '/overview',
    children: [
      {
        path: 'overview',
        name: 'overview',
        component: () => import('@/views/Overview.vue'),
        meta: { title: '总览', subtitle: '评测资产与运行状态一览' },
      },
      {
        path: 'datasets',
        name: 'datasets',
        component: () => import('@/views/dataset/List.vue'),
        meta: { title: '数据集' },
      },
      {
        path: 'datasets/:id',
        name: 'dataset-detail',
        component: () => import('@/views/dataset/Detail.vue'),
        meta: { title: '数据集详情' },
      },
      {
        path: 'models',
        name: 'models',
        component: () => import('@/views/model/List.vue'),
        meta: { title: '模型档案' },
```

```
    },
    {
      path: 'tasks',
      name: 'tasks',
      component: () => import('@/views/task/List.vue'),
      meta: { title: '评测任务' },
    },
    {
      path: 'tasks/:id',
      name: 'task-detail',
      component: () => import('@/views/task/Detail.vue'),
      meta: { title: '任务监控' },
    },
    {
      path: 'analysis',
      name: 'analysis',
      component: () => import('@/views/analysis/Index.vue'),
      meta: { title: '结果分析', subtitle: '指标对比、消融与失败诊断' },
    },
    {
      path: 'replay',
      name: 'replay',
      component: () => import('@/views/replay/Workbench.vue'),
      meta: { title: '轨迹回放', subtitle: '四区联动的逐帧回放工作台' },
    },
    {
      path: 'sim',
      name: 'sim',
      component: () => import('@/views/sim/Workbench.vue'),
      meta: { title: '交互式仿真', subtitle: '在场景里手动驾驶机器狗，实时看第一视角与轨迹' },
    },
    {
      path: 'reports',
      name: 'reports',
      component: () => import('@/views/report/Center.vue'),
      meta: { title: '报告中心', subtitle: '分析报告与指标矩阵导出' },
    },
    {
      path: 'backup-import',
      name: 'backup-import',
      component: () => import('@/views/system/BackupImport.vue'),
      meta: { title: '备份库导入', subtitle: '自动同步数据集、模型档案与评测资料' },
    },
    {
      path: 'settings',
      name: 'settings',
      component: () => import('@/views/system/Settings.vue'),
      meta: { title: '系统设置', subtitle: '运行参数与系统状态' },
    },
    {
```

```
        path: 'users',
        name: 'users',
        component: () => import('@/views/system/Users.vue'),
        meta: { title: '用户管理', admin: true },
      },
      {
        path: 'audit',
        name: 'audit',
        component: () => import('@/views/system/Audit.vue'),
        meta: { title: '操作审计', admin: true },
      },
    ],
  },
  { path: '/:pathMatch(.*)*', redirect: '/overview' },
]

const router = createRouter({
  history: createWebHistory(),
  routes,
})

router.beforeEach((to) => {
  const auth = useAuthStore()
  if (!to.meta.public && !auth.isLoggedIn) {
    return { path: '/login', query: { redirect: to.fullPath } }
  }
  if (to.meta.public && auth.isLoggedIn && to.path === '/login') {
    return { path: '/overview' }
  }
  if (to.meta.admin && auth.role !== 'admin') {
    return { path: '/overview' }
  }
  document.title = to.meta.title
    ? `${to.meta.title} · 码极客 NavEval Studio`
    : '码极客具身导航多模态流式评测与诊断平台 V1.0'
  return true
})

export default router
import axios from 'axios'
import { ElMessage } from 'element-plus'

export const TOKEN_KEY = 'naveval_token'
export const USER_KEY = 'naveval_user'

const http = axios.create({
  baseURL: '/api',
  timeout: 30000,
})
```

```
http.interceptors.request.use((config) => {
  const token = localStorage.getItem(TOKEN_KEY)
  if (token) {
    config.headers.Authorization = `Bearer ${token}`
  }
  return config
})

http.interceptors.response.use(
  (response) => {
    const body = response.data
    if (body && typeof body === 'object' && 'code' in body) {
      if (body.code !== 0) {
        ElMessage.error(body.message || '请求失败')
        return Promise.reject(new Error(body.message))
      }
      return body.data
    }
    return body
  },
  (error) => {
    const status = error.response?.status
    const body = error.response?.data
    const message = body?.message || error.message || '网络异常'
    // 轮询/操控这类高频请求用 silent: true：卡住时不要把页面刷成一墙红条
    const silent = Boolean(error.config?.silent)

    if (status === 401) {
      localStorage.removeItem(TOKEN_KEY)
      localStorage.removeItem(USER_KEY)
      if (!window.location.pathname.startsWith('/login')) {
        window.location.href = '/login'
      }
    } else if (!silent) {
      ElMessage.error(message)
    }
    return Promise.reject(new Error(message))
  },
)

export const api = {
  login: (payload) => http.post('/auth/login', payload),
  logout: () => http.post('/auth/logout'),
  me: () => http.get('/auth/me'),
  changePassword: (payload) => http.post('/auth/password', payload),

  overview: () => http.get('/overview'),
  systemStatus: () => http.get('/system/status'),
  gpus: () => http.get('/system/gpus'),
  settings: () => http.get('/settings'),
```

```
  updateSettings: (items) => http.put('/settings', { items }),

  users: () => http.get('/users'),
  createUser: (payload) => http.post('/users', payload),
  updateUser: (id, payload) => http.patch(`/users/${id}`, payload),
  audit: (params) => http.get('/audit', { params }),

  datasets: (params) => http.get('/datasets', { params }),
  createDataset: (payload) => http.post('/datasets', payload),
  datasetDetail: (id) => http.get(`/datasets/${id}`),
  updateDataset: (id, payload) => http.patch(`/datasets/${id}`, payload),
  deleteDataset: (id) => http.delete(`/datasets/${id}`),
  scanDataset: (id) => http.post(`/datasets/${id}/scan`),

  episodes: (id, params) => http.get(`/datasets/${id}/episodes`, { params }),
  episode: (id, rowId) => http.get(`/datasets/${id}/episodes/${rowId}`),
  observations: (id, rowId) => http.get(`/datasets/${id}/episodes/${rowId}/observations`),
  validateDataset: (id) => http.post(`/datasets/${id}/validate`),

  splits: (id) => http.get(`/datasets/${id}/splits`),
  createSplit: (id, payload) => http.post(`/datasets/${id}/splits`, payload),
  deleteSplit: (splitId) => http.delete(`/splits/${splitId}`),
  exportCausal: (id, payload) => http.post(`/datasets/${id}/export-causal`, payload),
  downloadExport: (id, filename) =>
    http.get(`/datasets/${id}/exports/${filename}`, { responseType: 'blob' }),

  models: () => http.get('/models'),
  createModel: (payload) => http.post('/models', payload),
  updateModel: (id, payload) => http.patch(`/models/${id}`, payload),
  deleteModel: (id) => http.delete(`/models/${id}`),
  probeModel: (id) => http.post(`/models/${id}/probe`),

  templates: () => http.get('/templates'),
  createTemplate: (payload) => http.post('/templates', payload),
  updateTemplate: (id, payload) => http.patch(`/templates/${id}`, payload),
  deleteTemplate: (id) => http.delete(`/templates/${id}`),

  tasks: (params) => http.get('/tasks', { params }),
  createTask: (payload) => http.post('/tasks', payload),
  taskDetail: (id) => http.get(`/tasks/${id}`),
  taskEpisodes: (id, params) => http.get(`/tasks/${id}/episodes`, { params }),
  taskLogs: (id, params) => http.get(`/tasks/${id}/logs`, { params }),
  taskSamples: (id, params) => http.get(`/tasks/${id}/samples`, { params }),
  cancelTask: (id) => http.post(`/tasks/${id}/cancel`),
  pauseTask: (id) => http.post(`/tasks/${id}/pause`),
  resumeTask: (id) => http.post(`/tasks/${id}/resume`),
  retryTask: (id) => http.post(`/tasks/${id}/retry-failed`),

  importSources: () => http.get('/import/sources'),
  importResults: (payload) => http.post('/import/results', payload, { timeout: 0 }),
```

```
  backupOverview: (params) => http.get('/import/backup', { params }),
  backupImport: (payload) => http.post('/import/backup', payload, { timeout: 0 }),
  updateBackupAuto: (payload) => http.put('/import/backup/auto', payload),
  backupAssets: (params) => http.get('/import/backup/assets', { params }),

  analysisTasks: (params) => http.get('/analysis/tasks', { params }),
  analysisCompare: (params) => http.get('/analysis/compare', { params }),
  analysisAblation: (params) => http.get('/analysis/ablation', { params }),
  analysisBootstrap: (payload) => http.post('/analysis/bootstrap', payload),
  analysisLongHorizon: (params) => http.get('/analysis/longhorizon', { params }),
  analysisFailures: (params) => http.get('/analysis/failures', { params }),

  replayEpisode: (taskId, rowId) => http.get(`/replay/${taskId}/${rowId}`),

  // 交互式仿真：画面走 <img src="/api/sim/stream?token=">，这里只封装 JSON 接口
  simStart: (payload) => http.post('/sim/sessions', payload, { timeout: 0 }),
  // 状态轮询与操控都要短超时：后端一旦卡住，长超时的请求会把浏览器的 6 条连接全占死，
  // 连"重新连接画面"都发不出去（页面表现为整页卡死）。silent 是为了不刷屏。
  simState: () => http.get('/sim/state', { timeout: 4000, silent: true }),
  simAct: (action) => http.post('/sim/actions', { action }, { timeout: 6000, silent: true }),
  simStop: (sessionId) => http.delete(`/sim/sessions/${sessionId || 'current'}`),
  // 模型驾驶：启用后权重在后台线程加载（约 12 s），这里不等加载完成
  simPolicyStart: (payload = {}) => http.post('/sim/policy', payload),
  simPolicyStop: () => http.delete('/sim/policy'),
  simPolicyPause: (paused) => http.post('/sim/policy/pause', { paused }),
  // Oracle 驾驶：沿 navmesh 最短路径自动走，用来和模型/人类轨迹对照
  simOracle: (enabled) => http.post('/sim/oracle', { enabled }),
  // 第三视角画中画（传感器在会话启动时按 sim.pip 建好，这里只切显隐）
  simPip: (enabled) => http.post('/sim/pip', { enabled }),
  // 机器狗模型显隐（模型在会话启动时按 sim.robot_mesh 加载）
  simRobot: (enabled) => http.post('/sim/robot', { enabled }),
  // 会话录制导出：启动编码后靠 /sim/state 的 recording.export 轮询进度，完成后拉 blob
  simExport: () => http.post('/sim/recordings/export'),
  simExportFile: () => http.get('/sim/recordings/export/file', { responseType: 'blob' }),

  reports: (params) => http.get('/reports', { params }),
  createReport: (payload) => http.post('/reports', payload),
  downloadReport: (id) => http.get(`/reports/${id}/download`, { responseType: 'blob' }),
}

export function apiErrorMessage(error) {
  return error?.response?.data?.message || error?.message || '请求失败'
}


export default http
import { defineStore } from 'pinia'
import { api, TOKEN_KEY, USER_KEY } from '@/api'
```

```
function readUser() {
  try {
    return JSON.parse(localStorage.getItem(USER_KEY) || 'null')
  } catch {
    return null
  }
}

export const useAuthStore = defineStore('auth', {
  state: () => ({
    token: localStorage.getItem(TOKEN_KEY) || '',
    user: readUser(),
  }),

  getters: {
    isLoggedIn: (state) => Boolean(state.token),
    displayName: (state) => state.user?.display_name || state.user?.username || '未登录',
    role: (state) => state.user?.role || 'viewer',
    isAdmin: (state) => state.user?.role === 'admin',
  },

  actions: {
    async login(form) {
      const data = await api.login(form)
      this.token = data.access_token
      this.user = data.user
      localStorage.setItem(TOKEN_KEY, data.access_token)
      localStorage.setItem(USER_KEY, JSON.stringify(data.user))
      return data.user
    },

    async fetchMe() {
      const user = await api.me()
      this.user = user
      localStorage.setItem(USER_KEY, JSON.stringify(user))
      return user
    },

    async logout() {
      try {
        if (this.token) await api.logout()
      } catch {
        // 注销失败不阻塞前端清理
      }
      this.token = ''
      this.user = null
      localStorage.removeItem(TOKEN_KEY)
      localStorage.removeItem(USER_KEY)
    },
  },
```

```
})
export const DATASET_KINDS = [
  { value: 'r2r_ce', label: 'R2R-CE' },
  { value: 'rxr_ce', label: 'RxR-CE' },
  { value: 'custom', label: '自定义' },
]

export function kindLabel(kind) {
  return DATASET_KINDS.find((item) => item.value === kind)?.label || kind || '未知'
}

export function datasetStatus(status) {
  const map = {
    ready: { label: '可用', type: 'success' },
    missing: { label: '路径缺失', type: 'danger' },
    invalid: { label: '格式异常', type: 'warning' },
    unknown: { label: '未扫描', type: 'info' },
  }
  return map[status] || { label: status || '未知', type: 'info' }
}

export function formatBytes(value) {
  const size = Number(value) || 0
  if (size < 1024) return `${size} B`
  if (size < 1024 * 1024) return `${(size / 1024).toFixed(1)} KB`
  return `${(size / 1024 / 1024).toFixed(2)} MB`
}

export function formatPercent(value, digits = 1) {
  if (value === null || value === undefined || Number.isNaN(Number(value))) return '—'
  return `${(Number(value) * 100).toFixed(digits)}%`
}

export function formatNumber(value, digits = 2) {
  if (value === null || value === undefined || Number.isNaN(Number(value))) return '—'
  return Number(value).toFixed(digits)
}

export function shortTime(value) {
  return value || '—'
}

const FAILURE_LABELS = {
  timeout: '超时',
  collision: '碰撞',
  off_track: '偏离轨迹',
  loop: '原地打转',
  stuck: '卡死',
  early_stop: '提前停止',
  goal_unreachable: '目标不可达',
```

```
  parse_error: '动作解析失败',
  oom: '显存不足',
  unknown: '未知',
}

export function failureLabel(reason) {
  return FAILURE_LABELS[reason] || reason || '未知'
}
<template>
  <div ref="container" class="chart" :style="{ height: `${height}px` }" />
</template>

<script setup>
import { onBeforeUnmount, onMounted, ref, watch } from 'vue'
import * as echarts from 'echarts'

const props = defineProps({
  option: { type: Object, required: true },
  height: { type: Number, default: 280 },
})

const container = ref(null)
let chart = null
let observer = null

function render() {
  if (!chart) return
  chart.setOption(props.option || {}, true)
}

onMounted(() => {
  chart = echarts.init(container.value)
  render()
  observer = new ResizeObserver(() => chart && chart.resize())
  observer.observe(container.value)
})

onBeforeUnmount(() => {
  if (observer) observer.disconnect()
  if (chart) chart.dispose()
  chart = null
})

watch(() => props.option, render, { deep: true })
</script>

<style scoped>
.chart {
  width: 100%;
}
```

```
</style>
<template>
  <div ref="container" class="chart" :style="{ height: `${height}px` }" />
</template>

<script setup>
import { onBeforeUnmount, onMounted, ref, watch } from 'vue'
import * as echarts from 'echarts'

const props = defineProps({
  xAxis: { type: Array, default: () => [] },
  series: { type: Array, default: () => [] },
  yName: { type: String, default: '' },
  height: { type: Number, default: 220 },
})

const container = ref(null)
let chart = null
let observer = null

function render() {
  if (!chart) return
  chart.setOption(
    {
      grid: { left: 52, right: 18, top: 30, bottom: 26 },
      tooltip: {
        trigger: 'axis',
        backgroundColor: 'rgba(15,23,42,0.9)',
        borderWidth: 0,
        textStyle: { color: '#e2e8f0', fontSize: 11 },
      },
      legend: {
        show: props.series.length > 1,
        right: 0,
        top: 0,
        itemWidth: 10,
        itemHeight: 10,
        textStyle: { color: '#64748b', fontSize: 11 },
      },
      xAxis: {
        type: 'category',
        data: props.xAxis,
        boundaryGap: false,
        axisLine: { lineStyle: { color: '#e5eaf2' } },
        axisTick: { show: false },
        axisLabel: { color: '#94a3b8', fontSize: 10 },
      },
      yAxis: {
        type: 'value',
        name: props.yName,
```

```
        nameTextStyle: { color: '#94a3b8', fontSize: 10 },
        splitLine: { lineStyle: { color: '#eef2f8' } },
        axisLabel: { color: '#94a3b8', fontSize: 10 },
      },
      series: props.series.map((item) => ({
        name: item.name,
        type: 'line',
        smooth: true,
        showSymbol: false,
        data: item.data,
        lineStyle: { width: 2, color: item.color },
        itemStyle: { color: item.color },
        areaStyle: item.area
          ? {
              color: new echarts.graphic.LinearGradient(0, 0, 0, 1, [
                { offset: 0, color: `${item.color}33` },
                { offset: 1, color: `${item.color}00` },
              ]),
            }
          : undefined,
      })),
    },
    true,
  )
}

onMounted(() => {
  chart = echarts.init(container.value)
  render()
  observer = new ResizeObserver(() => chart && chart.resize())
  observer.observe(container.value)
})

onBeforeUnmount(() => {
  if (observer) observer.disconnect()
  if (chart) chart.dispose()
  chart = null
})

watch(() => [props.xAxis, props.series], render, { deep: true })
</script>

<style scoped>
.chart {
  width: 100%;
}
</style>
<template>
  <div class="memory">
    <div class="legend">
```

```
      <span><i class="dot normal" />上下文长度</span>
      <span><i class="dot slow" />慢记忆更新</span>
      <span><i class="dot prune" />token 裁剪</span>
    </div>
    <div ref="container" class="chart" :style="{ height: `${height}px` }">
      <canvas ref="canvas" @click="onClick" />
    </div>
    <div class="ticks">
      <span>0</span>
      <span class="mono">{{ maxValue.toLocaleString() }} token</span>
      <span>{{ frames.length }} 帧</span>
    </div>
  </div>
</template>

<script setup>
import { computed, onBeforeUnmount, onMounted, ref, watch } from 'vue'

const props = defineProps({
  frames: { type: Array, default: () => [] },
  index: { type: Number, default: 0 },
  height: { type: Number, default: 74 },
})

const emit = defineEmits(['seek'])

const container = ref(null)
const canvas = ref(null)
let ctx = null
let observer = null
const maxValue = computed(() =>
  Math.max(1, ...props.frames.map((frame) => Number(frame.ctx_len || 0))),
)

function draw() {
  const rect = container.value?.getBoundingClientRect()
  if (!rect || !canvas.value) return
  const dpr = window.devicePixelRatio || 1
  const width = rect.width
  const height = rect.height
  canvas.value.width = Math.max(1, Math.round(width * dpr))
  canvas.value.height = Math.max(1, Math.round(height * dpr))
  ctx = canvas.value.getContext('2d')
  ctx.setTransform(dpr, 0, 0, dpr, 0, 0)
  ctx.clearRect(0, 0, width, height)

  const frames = props.frames
  if (!frames.length) return

  const bottom = height - 5
```

```
  const gap = frames.length > 90 ? 0 : 1
  const barWidth = Math.max(1, width / frames.length - gap)
  const scale = (bottom - 6) / maxValue.value

  frames.forEach((frame, i) => {
    const value = Number(frame.ctx_len || 0)
    const barHeight = Math.max(2, value * scale)
    const x = (i * width) / frames.length
    const y = bottom - barHeight
    const active = i === props.index
    if (active) {
      ctx.fillStyle = '#2f6fed'
    } else if (frame.slow_update) {
      ctx.fillStyle = '#f8c56b'
    } else {
      ctx.fillStyle = '#d8e6f9'
    }
    ctx.beginPath()
    ctx.roundRect(x, y, barWidth, barHeight, 2)
    ctx.fill()

    if (frame.tokens_after_prune < frame.tokens_before_prune) {
      ctx.fillStyle = '#ef8a8a'
      ctx.fillRect(x, y - 3, barWidth, 2)
    }
  })

  const [current] = [frames[props.index]]
  if (current) {
    const x = (props.index * width) / frames.length
    ctx.strokeStyle = '#2f6fed'
    ctx.lineWidth = 1.5
    ctx.beginPath()
    ctx.moveTo(x + barWidth / 2, 0)
    ctx.lineTo(x + barWidth / 2, bottom)
    ctx.stroke()
  }
}

function onClick(event) {
  if (!props.frames.length) return
  const rect = canvas.value.getBoundingClientRect()
  const ratio = (event.clientX - rect.left) / rect.width
  const index = Math.min(
    props.frames.length - 1,
    Math.max(0, Math.floor(ratio * props.frames.length)),
  )
  emit('seek', index)
}
```

```
onMounted(() => {
  draw()
  observer = new ResizeObserver(draw)
  observer.observe(container.value)
})

onBeforeUnmount(() => {
  if (observer) observer.disconnect()
  ctx = null
})

watch(() => [props.frames, props.index], draw, { deep: true })
</script>

<style scoped>
.memory {
  width: 100%;
}

.legend {
  display: flex;
  align-items: center;
  gap: 14px;
  font-size: 11px;
  color: #7d8fa8;
  margin-bottom: 6px;
}

.legend span {
  display: inline-flex;
  align-items: center;
  gap: 5px;
}

.dot {
  width: 7px;
  height: 7px;
  border-radius: 2px;
  display: inline-block;
}

.dot.normal {
  background: #d8e6f9;
}

.dot.slow {
  background: #f8c56b;
}

.dot.prune {
```

```
  background: #ef8a8a;
}

.chart {
  width: 100%;
  background: #fbfdff;
  border: 1px solid #edf2fa;
  border-radius: 6px;
}

canvas {
  width: 100%;
  height: 100%;
  display: block;
  cursor: pointer;
}

.ticks {
  display: flex;
  justify-content: space-between;
  margin-top: 4px;
  font-size: 10px;
  color: #9aa8bc;
}
</style>
<template>
  <div v-loading="loading" class="viewer">
    <div class="screen">
      <img
        v-if="src && !failed"
        :src="src"
        class="frame"
        alt="第一视角帧"
        @load="failed = false"
        @error="failed = true"
      />
      <div v-else class="frame-empty">
        <el-icon :size="24"><Picture /></el-icon>
        <span>{{ loading ? '加载帧图像…' : '当前 episode 没有可用帧' }}</span>
      </div>

      <img v-if="showMask && maskSrc" :src="maskSrc" class="mask" alt="可通行区域掩膜" />

      <div class="hud-top">
        <span class="hud-chip step">FRAME {{ (frame?.idx ?? 0) + 1 }} / {{ total || 0 }}</span>
        <span class="hud-chip action" :class="frame?.action || 'forward'">
          {{ actionLabel(frame?.action) }}
        </span>
        <span class="hud-chip mono">{{ latency }} ms</span>
        <span class="hud-chip mono">W{{ frame?.window_len ?? 0 }}</span>
```

```
      </div>

      <div v-if="!src && !loading" class="empty-hint">
        Mock 演示帧由服务端按逐帧元数据生成
      </div>

      <div class="hud-bottom">
        <div class="instruction">{{ instruction || '—' }}</div>
        <div class="hud-meta mono">
          x {{ poseText(0) }} · y {{ poseText(1) }} · z {{ poseText(2) }} · TOKENS
          {{ frame?.tokens_after_prune ?? 0 }}
          <span v-if="frame?.slow_update" class="slow">· 慢记忆更新</span>
        </div>
      </div>
    </div>
  </div>
</template>

<script setup>
import { computed, ref, watch } from 'vue'

const props = defineProps({
  src: { type: String, default: '' },
  maskSrc: { type: String, default: '' },
  frame: { type: Object, default: null },
  total: { type: Number, default: 0 },
  instruction: { type: String, default: '' },
  loading: { type: Boolean, default: false },
  showMask: { type: Boolean, default: false },
})

const failed = ref(false)
const latency = computed(() => Number(props.frame?.latency_ms || 0).toFixed(1))

const ACTION_LABELS = {
  forward: '前进',
  left: '左转',
  right: '右转',
  stop: '停止',
  turn_left: '左转',
  turn_right: '右转',
}

function actionLabel(action) {
  return ACTION_LABELS[action] || action || '未知'
}

function poseText(axis) {
  const value = props.frame?.pose?.[axis]
  return value === undefined ? '—' : Number(value).toFixed(2)
```

```
}

watch(
  () => props.src,
  () => {
    failed.value = false
  },
)
</script>

<style scoped>
.viewer {
  width: 100%;
  height: 100%;
}

.screen {
  position: relative;
  width: 100%;
  aspect-ratio: 16 / 9;
  max-height: 100%;
  border-radius: 8px;
  overflow: hidden;
  background: radial-gradient(circle at 50% 20%, #1b2c47, #0b1220 70%);
}

.frame {
  width: 100%;
  height: 100%;
  object-fit: cover;
  display: block;
}

.mask {
  position: absolute;
  inset: 0;
  width: 100%;
  height: 100%;
  object-fit: cover;
  opacity: 0.5;
  mix-blend-mode: screen;
  pointer-events: none;
}

.frame-empty {
  position: absolute;
  inset: 0;
  display: flex;
  flex-direction: column;
  align-items: center;
```

```
  justify-content: center;
  gap: 8px;
  color: #6f89ab;
  font-size: 12px;
}

.hud-top {
  position: absolute;
  top: 12px;
  left: 12px;
  right: 12px;
  display: flex;
  align-items: center;
  gap: 8px;
  pointer-events: none;
}

.hud-chip {
  padding: 3px 8px;
  border-radius: 5px;
  background: rgba(9, 17, 30, 0.68);
  border: 1px solid rgba(120, 155, 200, 0.22);
  color: #dbe7f7;
  font-size: 11px;
  letter-spacing: 0.4px;
  backdrop-filter: blur(4px);
}

.hud-chip.step {
  font-family: 'SFMono-Regular', Consolas, monospace;
}

.hud-chip.action {
  color: #08111f;
  font-weight: 600;
  border: none;
}

.hud-chip.action.forward {
  background: #6aa7ff;
}

.hud-chip.action.left {
  background: #4ade80;
}

.hud-chip.action.right {
  background: #fbbf24;
}
```

```
.hud-chip.action.stop {
  background: #cbd5e1;
}

.hud-bottom {
  position: absolute;
  left: 0;
  right: 0;
  bottom: 0;
  padding: 32px 16px 12px;
  background: linear-gradient(180deg, transparent, rgba(5, 11, 20, 0.88));
  pointer-events: none;
}

.instruction {
  color: #eaf2ff;
  font-size: 13px;
  line-height: 1.55;
  max-width: 92%;
}

.hud-meta {
  margin-top: 6px;
  color: #8fa8c7;
  font-size: 11px;
}

.slow {
  color: #fbbf24;
}

.empty-hint {
  position: absolute;
  left: 12px;
  bottom: 66px;
  color: #5f7695;
  font-size: 11px;
}
</style>
```